

Database Systems

Module 1 - Database Systems Concepts

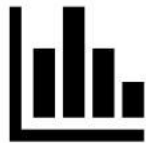
Dr. Shivani Gupta
Associate Professor
SCOPE, VIT Chennai



Topics to be covered

- Need for database systems
- Characteristics of Database Approach
- Advantages of using DBMS approach
- Actors on the Database Management Scene: Database Administrator
- Classification of database management systems
- Data Models
- Schemas and Instances
- Three-Schema Architecture
- The Database System Environment
- Centralized and Client/Server Architectures for DBMSs
- Overall Architecture of Database Management Systems

INTRODUCTION TO DBMS



What is DBMS?

- DATA ?
- ✓ It is a Latin word
- ✓ In layman's language, it is a collection of raw facts and figures which can be recorded
- ✓ Examples: Text, numbers, special symbols, images, videos, speech, etc.



INFORMATION?

- ✓ Processed data
- ✓ Plays a key role to decision making
- ✓ Example:

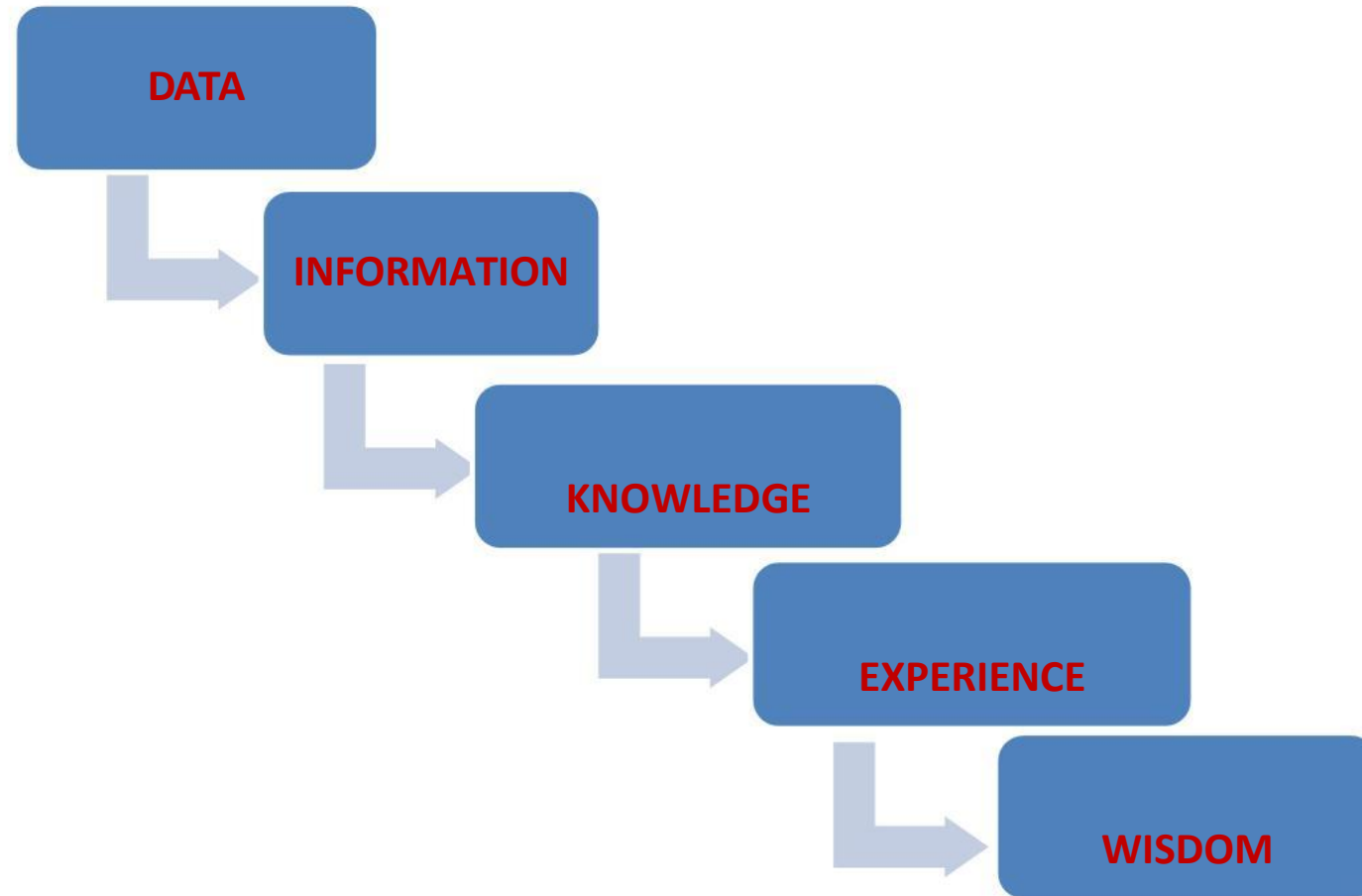


Data->5

Data->\$

5\$->an information

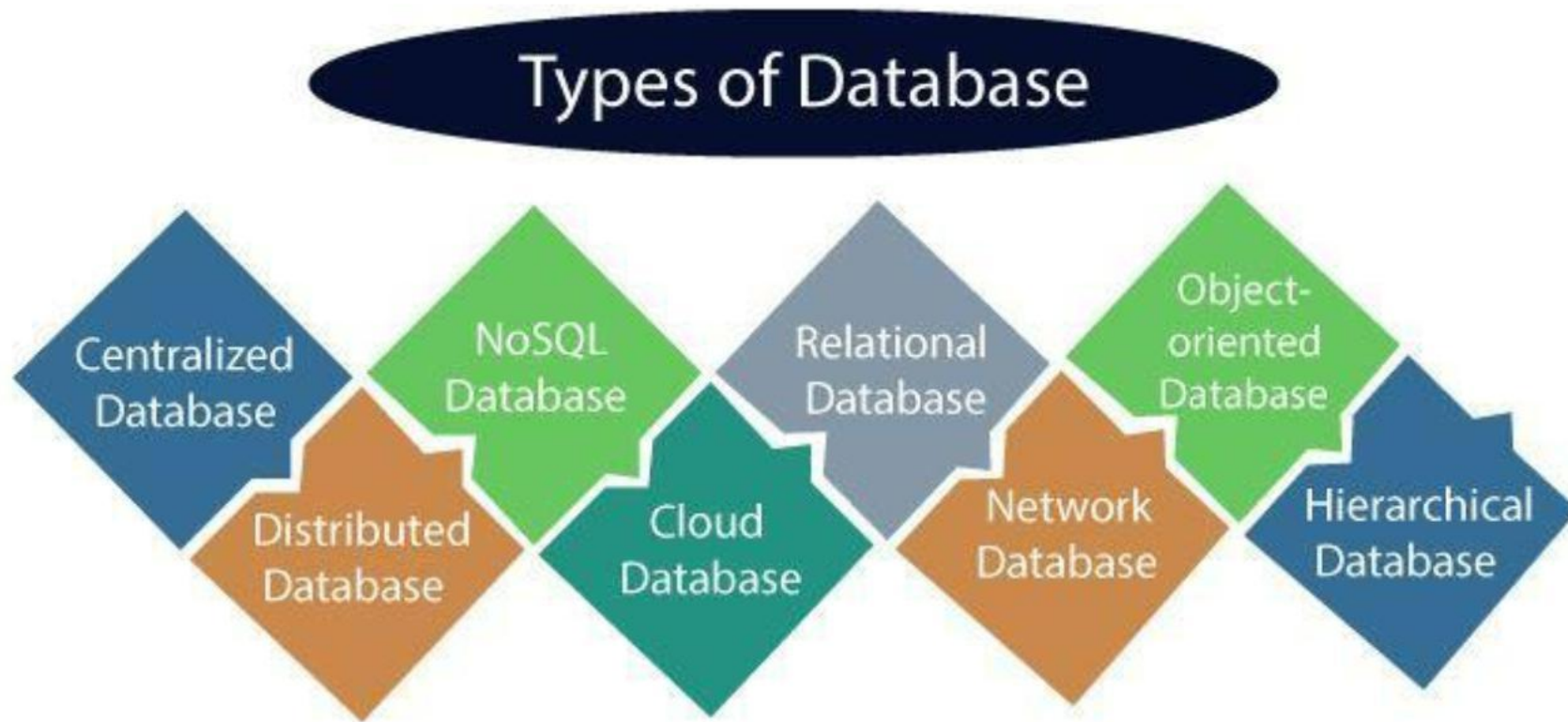
Derivation?



DATABASE

- Collection of related data
- Systematic and organized for effective and efficient retrieval and modification of data

Types of databases



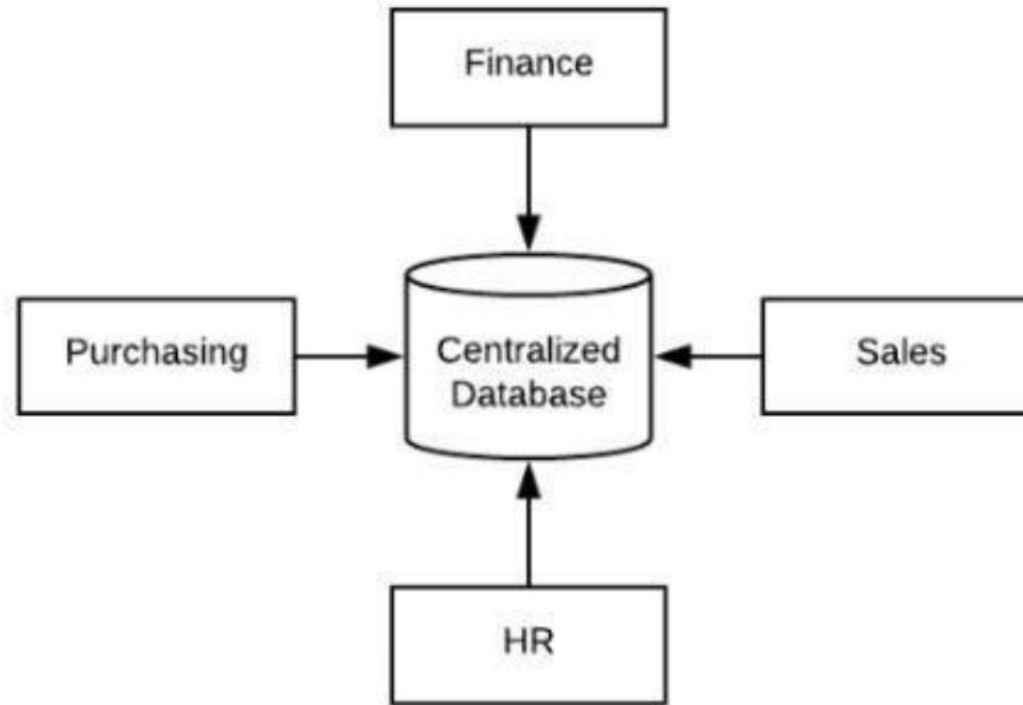
Types of databases

1) Centralized Database

It is the type of database that stores data at a centralized database system. It comforts the users to access the stored data from different locations through several applications. These applications contain the authentication process to let users access data securely. An example of a Centralized database can be Central Library that carries a central database of each library in a college/university.

Types of databases

1) Centralized Database: example



Types of databases

2) Distributed Database

Unlike a centralized database system, in distributed systems, data is distributed among different database systems of an organization. These database systems are connected via communication links. Such links help the end-users to access the data easily. Examples of the Distributed database are Apache Cassandra, HBase, etc.

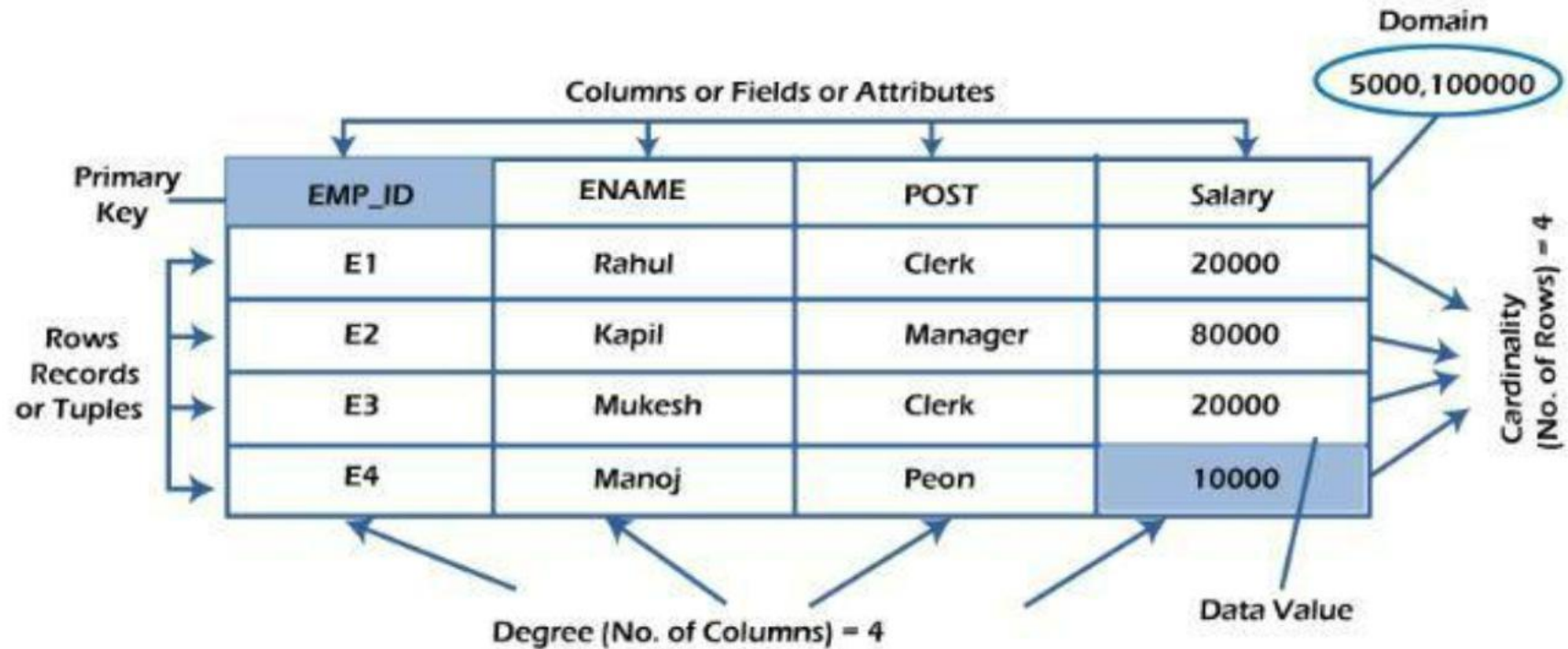
Types of databases

3) Relational Database

- This database is based on the relational data model, which stores data in the form of rows(tuple) and columns(attributes), and together forms a table(relation).
- A relational database uses SQL for storing, manipulating, as well as maintaining the data. E.F. Codd invented the database in 1970.
- Each table in the database carries a key that makes the data unique from others. Examples of Relational databases are MySQL, Microsoft SQL Server, Oracle, etc.

Types of databases

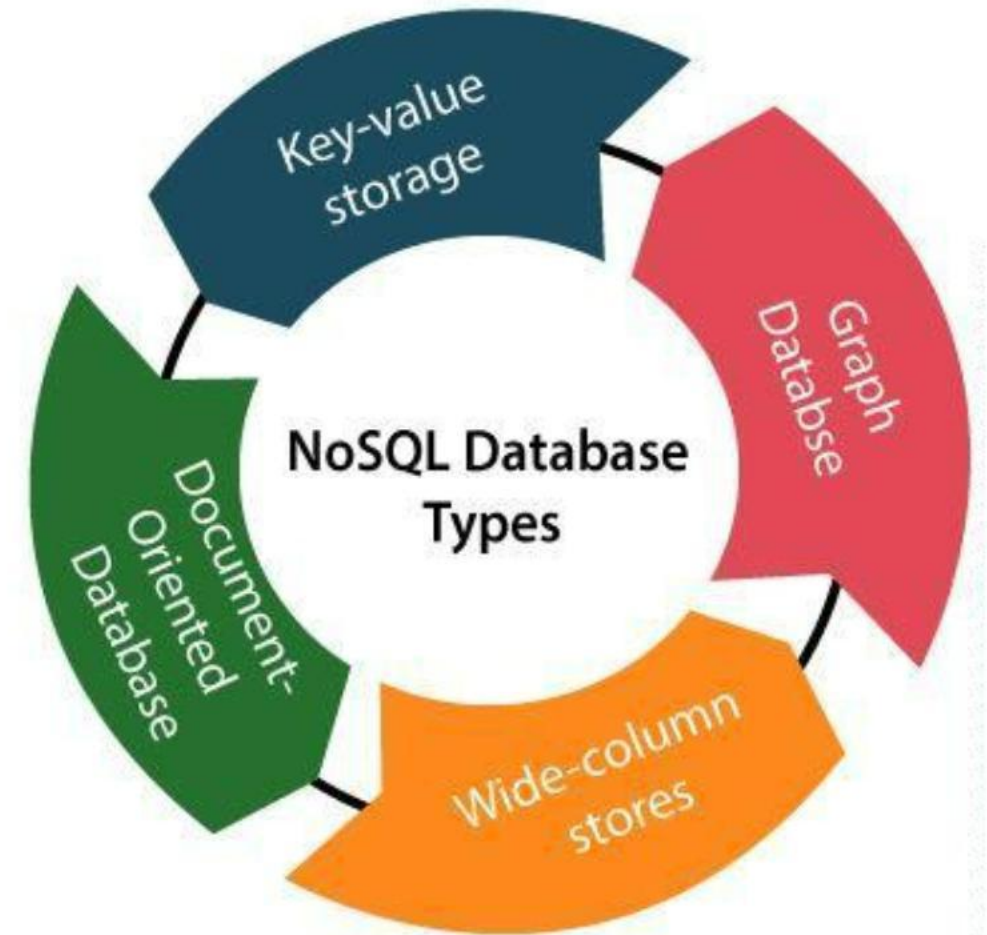
3) Relational Database: example



Types of databases

4) NoSQL Database

Non-SQL/Not Only SQL is a type of database that is used for storing a wide range of data sets. It is not a relational database as it stores data not only in tabular form but in several different ways.



Types of databases

5) Cloud Database

A type of database where data is stored in a virtual environment and executes over the cloud computing platform. It provides users with various cloud computing services (SaaS, PaaS, IaaS, etc.) for accessing the database. There are numerous cloud platforms, but the best options are:

- Amazon Web Services(AWS)
- Microsoft Azure
- Google Cloud SQL, etc.

Types of databases

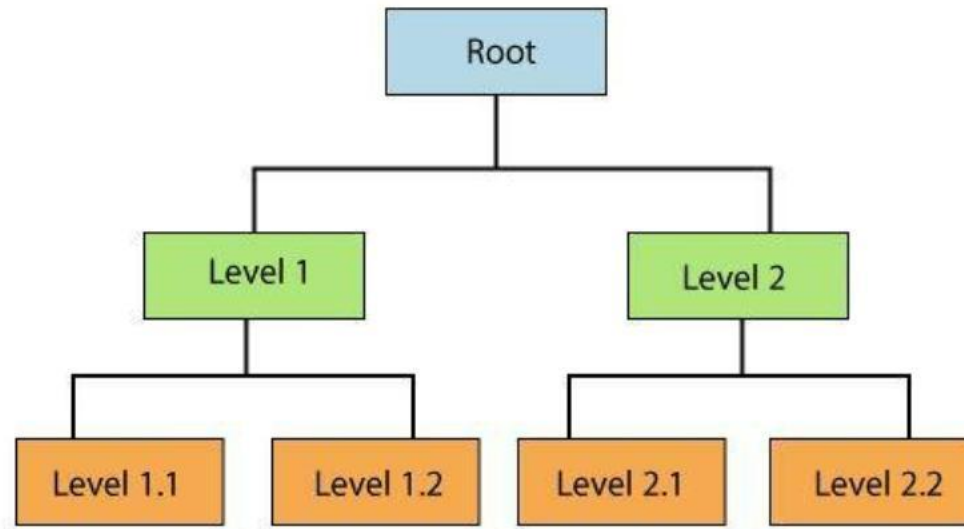
6) Object-oriented Database

The type of database that uses the object-based data model approach for storing data in the database system. The data is represented and stored as objects which are similar to the objects used in the object-oriented programming language.

Types of databases

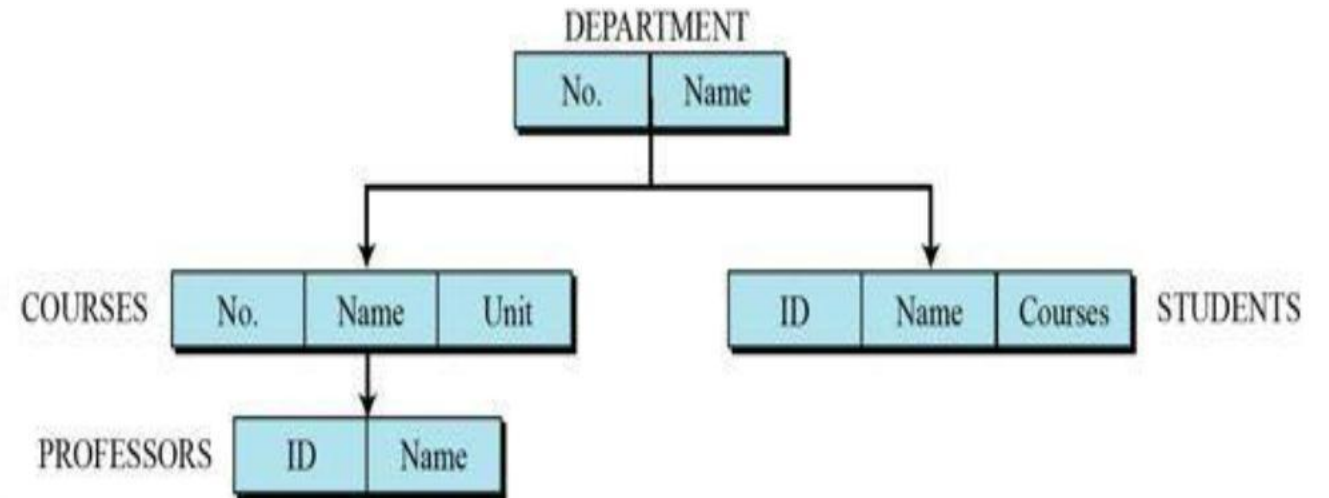
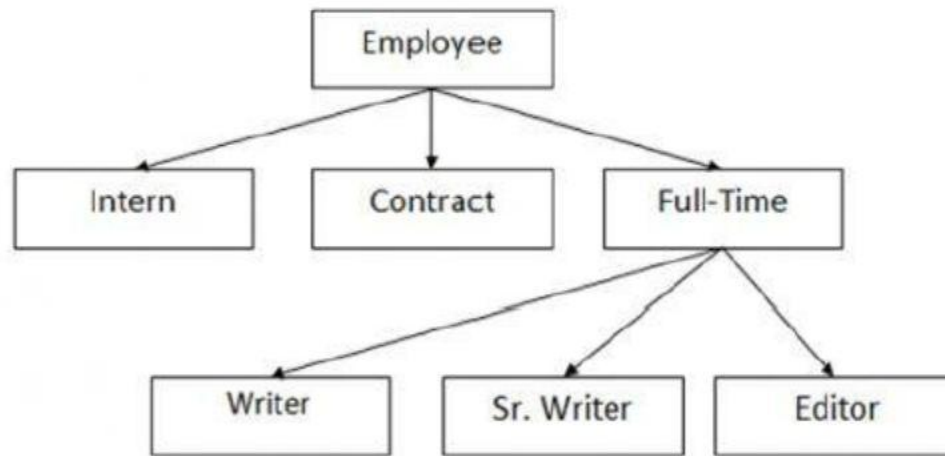
7) Hierarchical Database

It is the type of database that stores data in the form of parent-children relationship nodes. Here, it organizes data in a tree-like structure.



Types of databases

7) Hierarchical Database: example



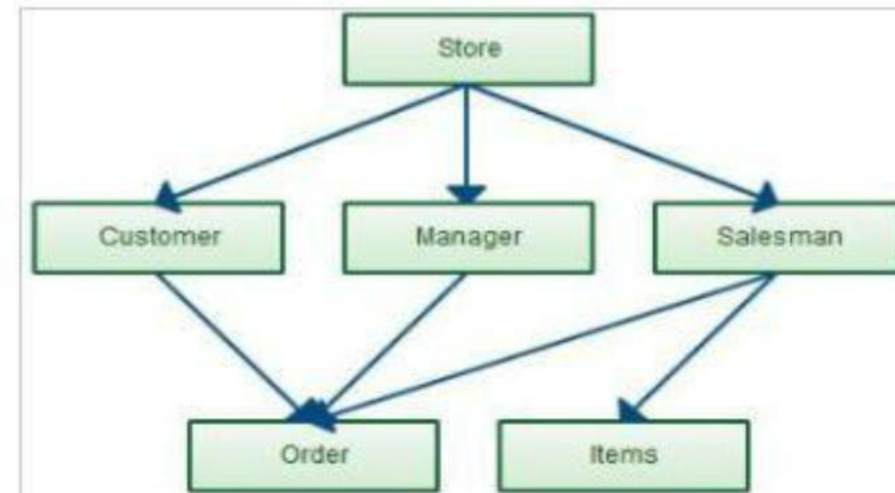
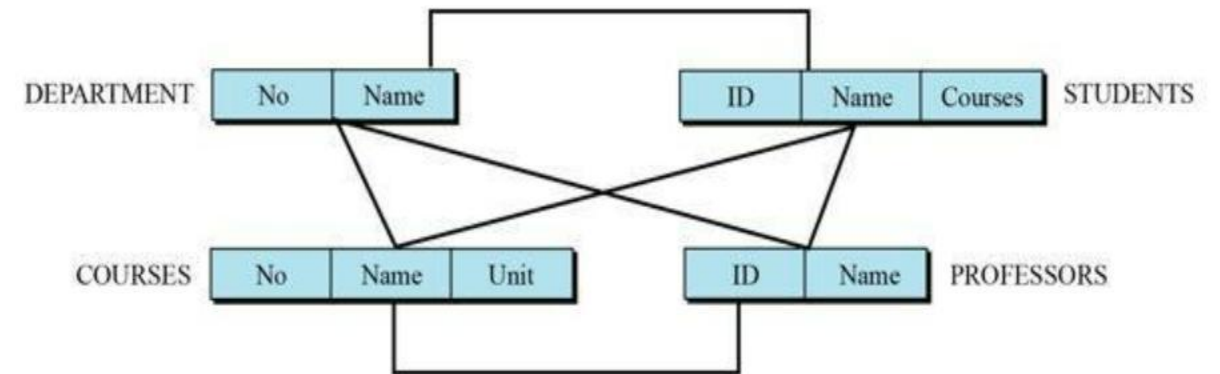
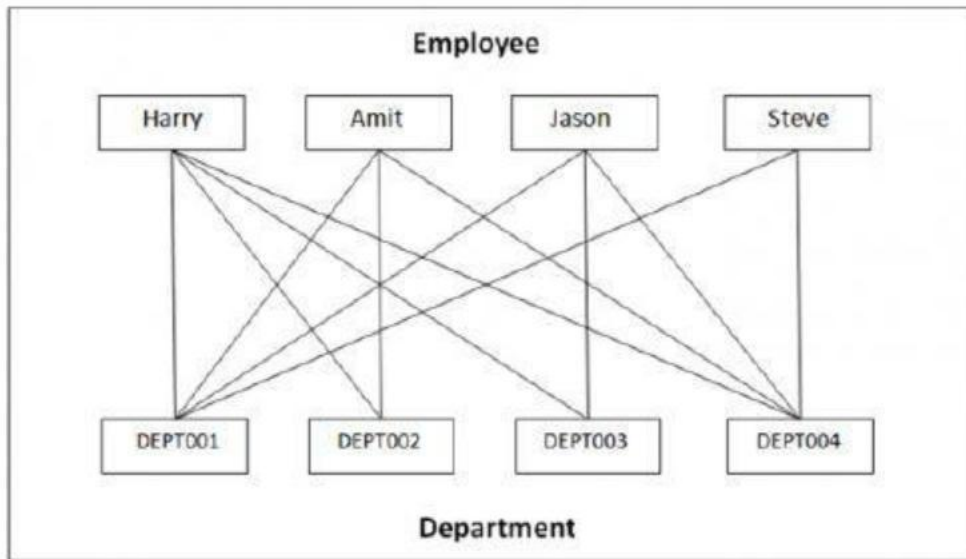
Types of databases

8) Network Database

It is the database that typically follows the network data model. Here, the representation of data is in the form of nodes connected via links between them. Unlike the hierarchical database, it allows each record to have multiple children and parent nodes to form a generalized graph structure.

Types of databases

8) Network Database: example



Examples:

- ✓ Manufacturing
- ✓ Sales
- ✓ Banking system
- ✓ University
- ✓ Airline

PERSISTENT DATA



DATABASE MANAGEMENT SYSTEM

- ✓ A DBMS is a database plus some programs that help us to manage that database. So, DBMS is software.
- ✓ We can say that a Database Management System or DBMS is a collection of logically related data and the programs that are used to modify that data.
- ✓ Some of the examples of the most commonly used DBMS are MySQL, Oracle, PostgreSQL, etc.

Why to study databases?

✓ Search

- Google
- Yahoo
- Bing



DATABASE MANAGEMENT SYSTEM

- Education

✓ Literature



Science



✓ Nursing



General topics



***SPACE**

***TIME**

History of DBMS



- A set of programs that
 - Define
 - Construct
 - Manipulate
-
- In the 1960's the first DBMS was developed at IBM by Charles W. Bachman.
 - Charles Bachman was an industrial researcher who developed a DBMS known as the Integrated Database Store (IDS). The system was standardized by CODASYL (Conference on Data Systems Languages).

History of DBMS



- Then, in 1970, E. F. Codd wrote “A Relational Model of Data for Large Shared Databases” and introduced the relational model, RDBMS.
- Popular DBMS
 - Oracle
 - MySQL
 - IBM DB2
 - Microsoft SQL Server
 - Sybase



Characteristics of DBMS (Database Management System)

1. **Real World Entity**: One of the most important and easy-to-understand characteristics of DBMS is that it is realistic. For example, we can have a Database Management System for a School or a big MNC, and the data is stored in the form of real-world entities. Any student that is stored in a Student database is like a real-world student (object/entity) and has properties (commonly known as attributes in DBMS terms) like his/her name, gender, age, roll number, etc.

Characteristics of DBMS (Database Management System)

2. Self Explaining Nature: A DBMS contains one database and along with that it also contains metadata about that one database. Metadata is the data about data.

For example: In a DBMS for a particular School, the total Number of rows in the database and what is the name of each column of the database table, and all such information about the data is metadata.

S_ID	Name	Age	Marks

Metadata
Number of Rows = 200
First Col = S ID
Second Col = Name
Third Col = Age
Fourth Col = Marks

Characteristics of DBMS (Database Management System)

3. Atomicity of Operations (Transactions): Atomicity means that either the operation should not be performed or it should be performed in its entirety i.e. it must be either 0% or 100% completed.

DBMS provides us with atomic operations (i.e. any operation which is either complete 100% or not complete at all). This is very useful and important.

Characteristics of DBMS (Database Management System)

4. Concurrent Access without Anomalies: Multiple users can access the database at the same time without any anomalies (problems: like 2 people trying to access the same data might restrict one from accessing it). For instance: Again, let us consider the database of a bank where a person Rohan has his account. Let us say that Rohan has a brother Neil and he gives Neil his ATM card to withdraw some cash from the ATM. Now, when Neil reached the ATM, at the same time Rahul asked Rohan to transfer 1000 Rupees to him. If Rohan's initial balance was rupees 5000 and he transfers rupees 1000 to Rahul and he asks Neil to withdraw 2000 Rupees, his balance should now be Rupees 2000 ($5000 - 3000$). If the users would not have concurrent access to the database, either Neil's transaction or Rohan's transaction would have taken place and instead of 3000 Rupees, only 1000 or 2000 rupees would have been less from Rohan's account.

Characteristics of DBMS (Database Management System)

5. Integrity: Integrity means that the data which comes into the Database should be correct as well as consistent. Let us see an example to understand the meaning of correct and consistent data with respect to DBMS.

6. Ease of Access (The DBMS Queries): Before DBMS, the conventional file system (i.e. files and folders) was used to store complex and large data. Searching for a Name in thousands of Students was a very difficult task. This is because manual searching is done in a conventional file system for a particular data. However, DBMS provides ease of access to the data inside the database. We can run a search query to find any data and the process is way faster than manual searching and is more reliable.

Characteristics of DBMS (Database Management System)

7. SQL and No-SQL Databases: There are 2 types of databases (not DBMS): SQL

The SQL databases store the data in the form of Tables i.e. rows and columns. The No-SQL databases can store data in any other form than a table. For instance: the very popular MongoDB stores the data in the form of JSON (JavaScript Object Notation). The availability of SQL and No-SQL databases provides us with the flexibility of choosing the method of storing the data as well. This is a characteristic of DBMS because DBMS allows us to perform operations on both kinds of databases. So, we can run queries and operations on SQL as well as No-SQL databases.

Characteristics of DBMS (Database Management System)

8. ACID Properties: The DBMS follows certain properties to maintain consistency in the database. These properties are usually termed as ACID Properties. ACID stands for Atomicity, Consistency, Isolation, and Durability. Atomicity means the transaction should either be 0% or 100% completed and consistency means that the change in data should be reflected everywhere in a database. Isolation means that multiple transactions can occur independently without the interference of some other transactions. Durability means that the chances of a successful atomic transaction i.e. a transaction that has been 100% completed should reflect in the database.

Characteristics of DBMS (Database Management System)

9. Security: The database should be accessible to the users in a limited way. The access to make changes to a database by the user should be limited and the users must not be given complete access to the entire database. Unauthorized users should not be allowed to access the database.

Authentication: The DBMS has authentication for various users that directly refers to the limit to which the user can access the Database. Authentication means the process of logging in of the user only with the rights that he/she has been authorized to. For instance, in any organization, admin has the access to make changes to the database of the organization as some new employee might have joined the organization or someone might have left it. However, the employees have access only to their personal profile and can make changes to it only. They cannot access the database of any other employee or the organization as a whole.



Issues with traditional file management system

- Data redundancy
- Data inconsistency
- Difficulty in accessing data
- Data isolation— multiple files and formats
- Security problems
- Atomicity problem
- Concurrent access anomalies
- Integrity problems

DBMS vs File systems:

Basis	DBMS Approach	File System Approach
Meaning	DBMS is a collection of data. In DBMS, the user is not required to write the procedures.	The file system is a collection of data. In this system, the user has to write the procedures for managing the database.
Sharing of data	Due to the centralized approach, data sharing is easy.	Data is distributed in many files, and it may be of different formats, so it isn't easy to share data.
Data Abstraction	DBMS gives an abstract view of data that hides the details.	The file system provides the detail of the data representation and storage of data.
Security and Protection	DBMS provides a good protection mechanism.	It isn't easy to protect a file under the file system.
Recovery Mechanism	DBMS provides a crash recovery mechanism, i.e., DBMS protects the user from system failure.	The file system doesn't have a crash mechanism, i.e., if the system crashes while entering some data, then the content of the file will be lost.
Manipulation Techniques	DBMS contains a wide variety of sophisticated techniques to store and retrieve	The file system can't efficiently store and retrieve the data.

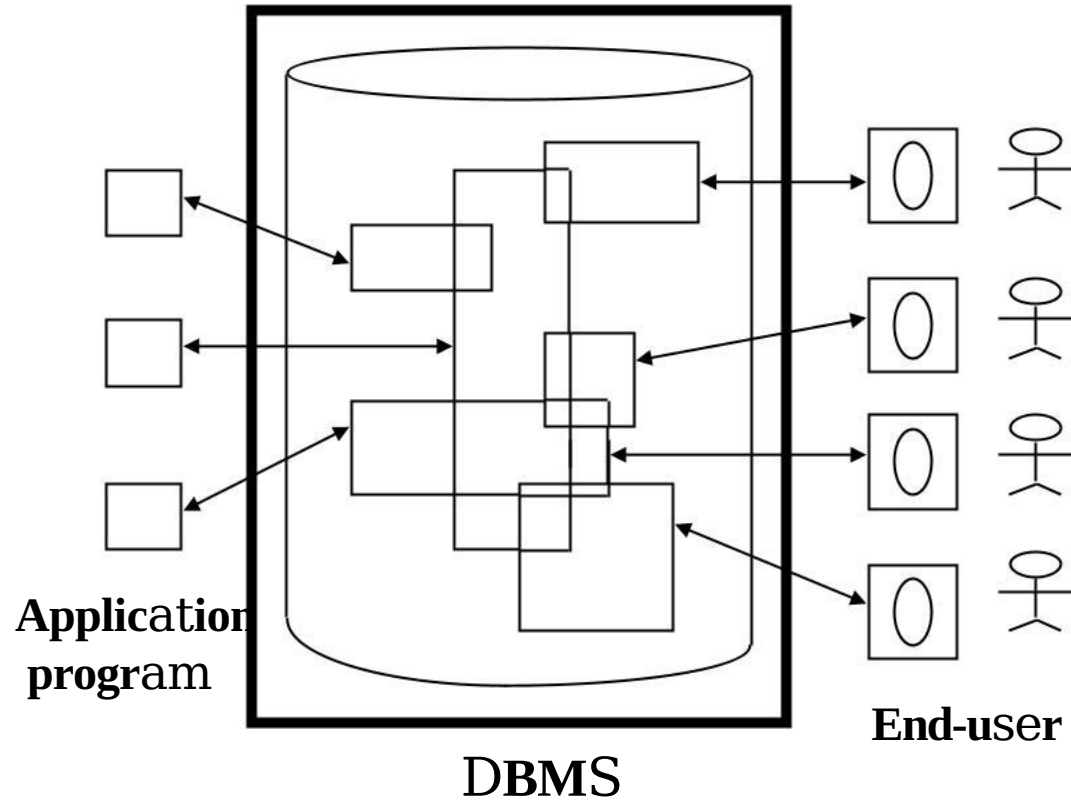
DBMS vs File systems contd...

Concurrency Problems	DBMS takes care of Concurrent access of data using some form of locking.	In the File system, concurrent access has many problems like redirecting the file while deleting some information or updating some information.
Where to use	Database approach used in large systems which interrelate many files.	File system approach used in large systems which interrelate many files.
Cost	The database system is expensive to design.	The file system approach is cheaper to design.
Data Redundancy and Inconsistency	Due to the centralization of the database, the problems of data redundancy and inconsistency are controlled.	In this, the files and application programs are created by different programmers so that there exists a lot of duplication of data which may lead to inconsistency.
Structure	The database structure is complex to design.	The file system approach has a simple structure.
Data Independence	In this system, Data Independence exists, and it can be of two types. ◦ Logical Data Independence ◦ Physical Data Independence	In the File system approach, there exists no Data Independence.
Integrity Constraints	Integrity Constraints are easy to apply.	Integrity Constraints are difficult to implement in file system.

Need Of DBMS

1. Ease of Accessing Data
2. Storage and Management of Data
3. Easy and Efficient File Management
4. Avoiding duplicates and Redundancy
5. Concurrent Data Accessing
6. Database Integrity

Architecture of DBMS





Essential components of a database system?

- ✓ Data
- ✓ Hardware (disk, CPU, main memory, etc.)
- ✓ Software
- ✓ Users

Database users?

Users may be divided into

- Those who actually use and control the database content, and those who design, develop and maintain database applications (called “Actors on the Scene”), and
- Those who design and develop the DBMS software and related tools, and the computer systems operators (called “Workers Behind the Scene”).

Database users?

Actors on the scene

Database administrators:

Responsible for authorizing access to the database, for coordinating and monitoring its use, acquiring software and hardware resources, controlling its use and monitoring efficiency of operations.

Database Designers:

Responsible to define the content, the structure, the constraints, and functions or transactions against the database. They must communicate with the end-users and understand their needs.

Database users?

Actors on the scene (continued)

End-users: They use the data for queries, reports and some of them update the database content. End-users can be categorized into:

Casual: access database occasionally when needed

Naïve or Parametric: they make up a large section of the end-user population.

They use previously well-defined functions in the form of “canned transactions” against the database.

Examples are bank-tellers or reservation clerks who do this activity for an entire shift of operations.

Database users?

Sophisticated:

These include business analysts, scientists, engineers, others thoroughly familiar with the system capabilities.

Many use tools in the form of software packages that work closely with the stored database.

Stand-alone:

Mostly maintain personal databases using ready-to-use packaged applications.

An example is a tax program user that creates its own internal database.

Another example is a user that maintains an address book

Database users?

Application programmers: **Application Programmers** also known as **Back-End Developers**, are computer professional users who are responsible for developing the **application programs** (C, C++, Java, PHP, Python, etc.) or the user interface so that other users can use these applications to interact with the database.

Application Programmers have deep knowledge of DBMS & databases and know everything in detail. They interact with the database using **DML (Data Manipulation Language)** queries to store data inside the database and when needed, they can also fetch the data from it.

Database users?

✓ End users

--**Naïve**: Naive users also known as **Parametric End users**, don't have any knowledge of DBMS but still frequently use the database applications to get the desired results. With the help of the interface provided by the DBMS applications, **Naive users** mostly use the database to fill in or retrieve the information (view level of the database).

Naive users don't need to be aware of the presence of the database system as they can interact with the database with the help of **menu-driven** application interface. In simple terms, **Naive / Parametric End users** work directly on developed applications to access the database indirectly for getting the desired results.

For example: Railway ticket booking users in general are naive/parametric end users as they don't have much knowledge about DBMS and directly use railway booking application to book their tickets.

Database users?

✓ End users

--**Sophisticated**: Sophisticated users are the type of database users in DBMS who know **DBMS (DDL & DML commands)** and are familiar with the database. Sophisticated users can be business analysts, engineers, scientists, system analysts, etc.

Sophisticated users can develop or access their database applications according to the requirements, without actually writing the program code for it. **These users are also known as SQL programmers** as they can interact with the database directly using SQL queries using query processors. Using SQL queries, they can fetch the data from the database. They can also delete, update or insert new data into the database.

For example: Data Engineers & Developers who are familiar with the database, directly access the database using SQL queries rather than writing programming code for accessing the database again. Hence, they are termed as sophisticated users.

Database users?

✓ End users

--**Specialized:** Specialized users are also called sophisticated users but they are responsible for writing the special database application programs. The specialized users are responsible to develop the programs according to the given requirements.



Database users?

- ✓ DBA: Short for database administrator, a DBA designs, implements, administers, and monitors data management systems and ensures design, consistency, quality, and security.

Role of DBA?

- ✓ Schema definition
- ✓ Storage structure and access method definition
- ✓ Schema and physical organization modification
- ✓ Granting user authority to access the database
- ✓ Monitoring performance and responding to changes in requirements

View of Data

❑ For the ease of understanding, the system hides the background details, i.e. the details of storage and organization of data. How the data is maintained, etc. Thus, it provides an abstract view of data.

- Levels of Abstraction
- Instances and Schemas
- Data Models

Levels of Abstraction

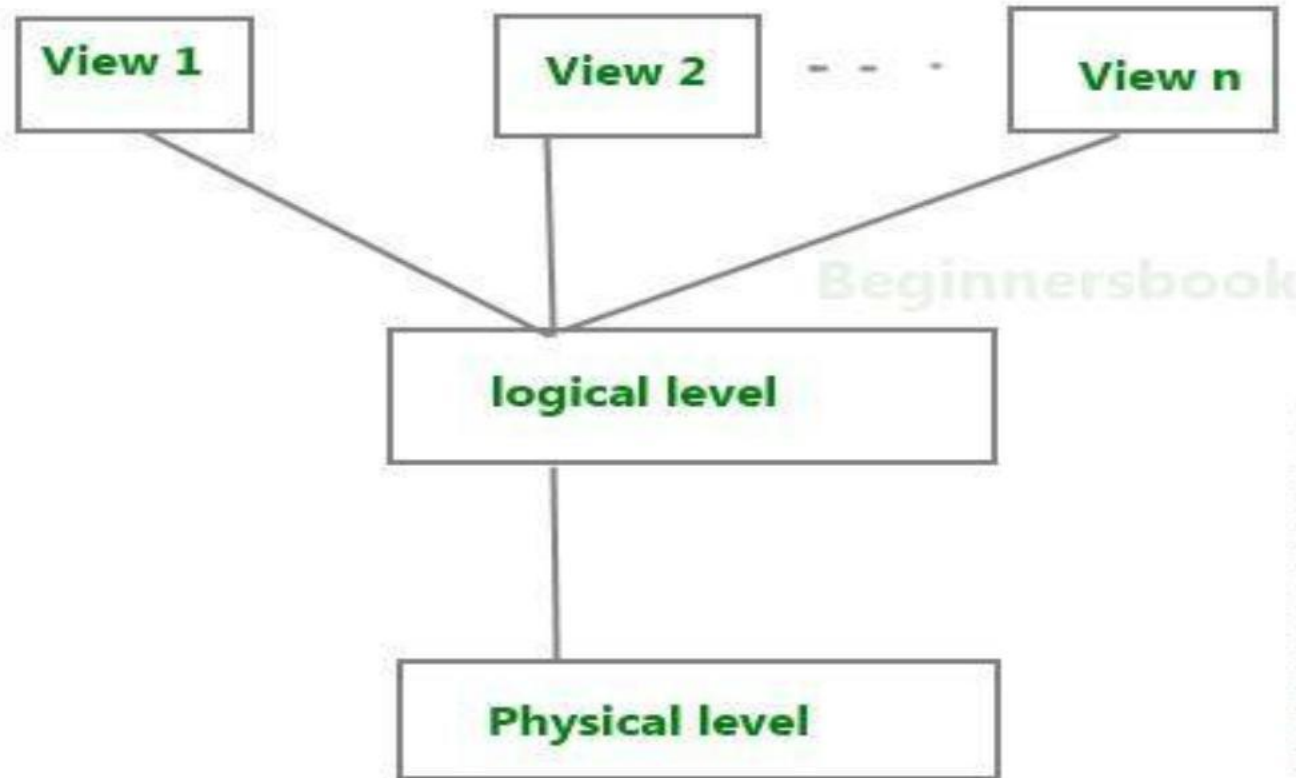
- The main purpose is to retrieve data easily and efficiently.
- Many database users are not computer trained. So, the developers hide complexity from users through several levels of abstraction to simplify the interactions of users with the system.
- Thus, different users can view data from different levels of details or abstraction needed.



Levels of Abstraction

- ✓ **Physical Level:** Lowest level of abstraction that describes how data is actually stored. The details are often hidden from programmers by the database system. DBA may be aware of certain details of physical organization of data.
- ✓ **Logical Level:** Next higher level of abstraction shows what data is stored. Both database programmers and DBA work at this level.
- ✓ **View Level:** Highest level of abstraction. Through GUI.

Levels of Abstraction



Three Levels of data abstraction

Instance and Schema

Instances?

- ✓ Collection of data or information stored at a particular moment of time.
- ✓ Can frequently change over time.

Schemas?

Overall design of a database

- ✓ Skeleton of a database
 - Physical schema at physical level,
 - Logical schema at logical level, and
 - sub-schema at view level.

Instance and Schema

Data independence:

- Logical data independence
- Physical data independence

Physical data independence

It is mainly used when we want to separate the conceptual level data from the physical/ internal levels. Physical data independence is fairly easy to achieve. It helps a user change the storage structure physically of the devices that have a direct effect on their conceptual schema.

Here are a few examples of changes made under the physical data independence:

- Modification of indexes.
- Altering the access method.
- The process of switching to a different data structure.
- Modification of the file organization technique in a DB (Database).
- Using some new storage device, such as Magnetic Tapes or Hard Drive.
- For altering the hashtag algorithms or compression techniques.
- Changing the database location. For example, from C to D drive.

Logical Data Independence

It is mainly used when we want to change the conceptual scheme, but we don't want to change the External programs or APIs and external views.

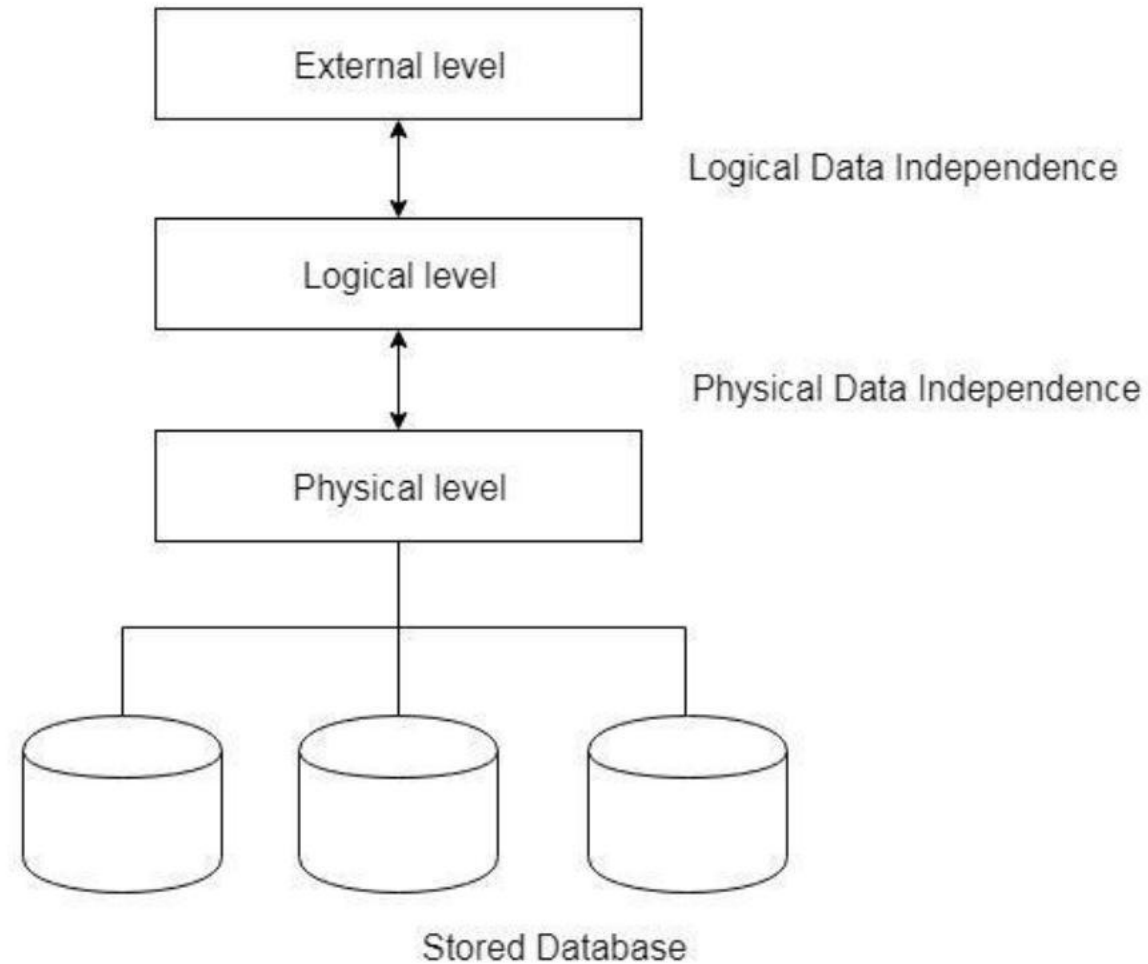
Here are a few examples of changes made under the Logical Data Independence:

- For deleting, modifying, or adding a new attribute. This relationship or entity is possible even if we don't rewrite the existing apps or programs.
- When we are merging two of the records into a single one.
- When we are trying to break an existing record. Meaning, when we are dividing any record into further two records.

Importance of Data Independence

- Helps you to improve the quality of the data
- Database system maintenance becomes affordable
- Enforcement of standards and improvement in database security
- You don't need to alter data structure in application programs
- Permit developers to focus on the general structure of the Database rather than worrying about the internal implementation
- It allows you to improve state which is undamaged or undivided
- Easily make modifications in the physical level is needed to improve the performance of the system

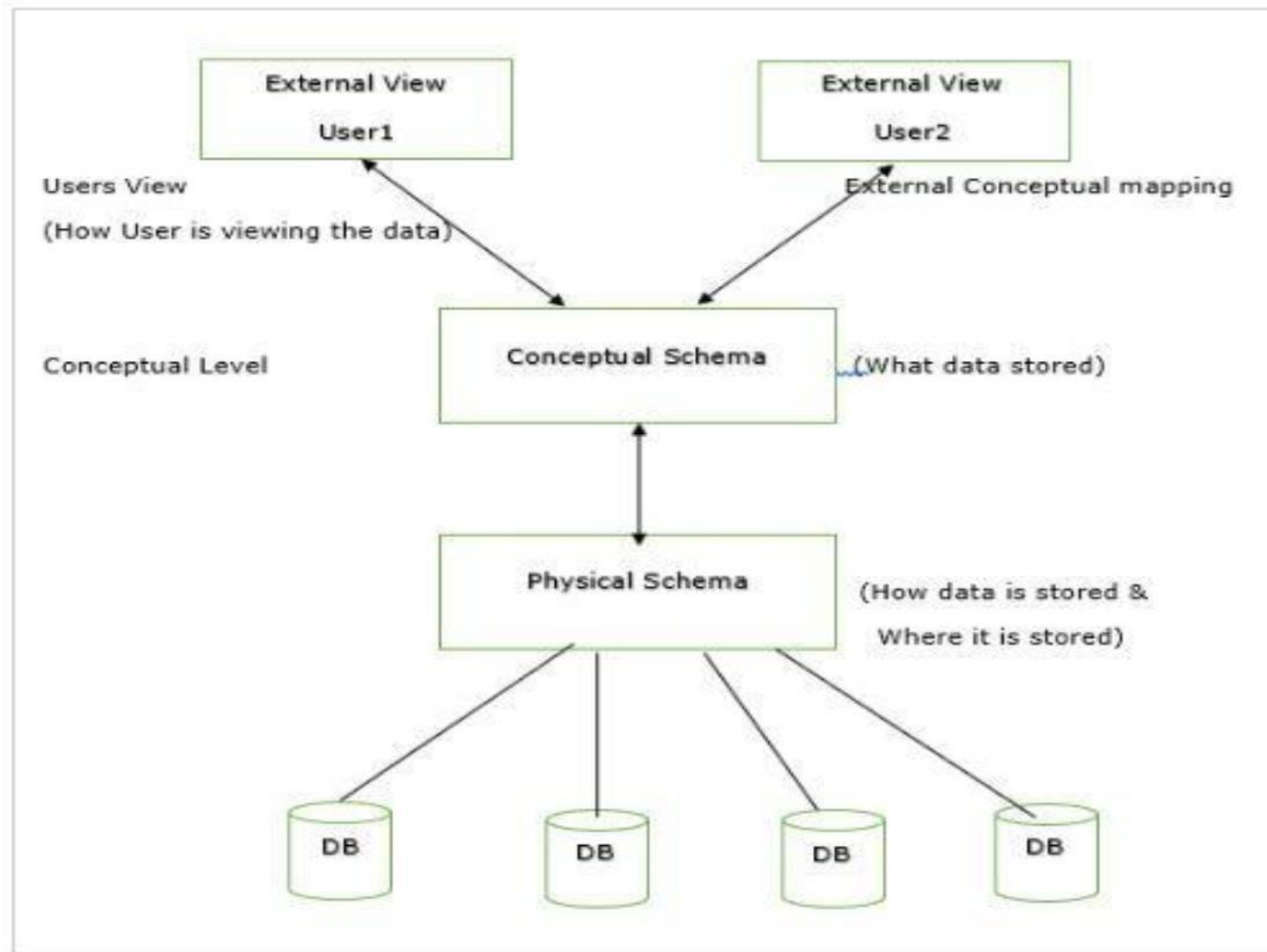
Data Independence



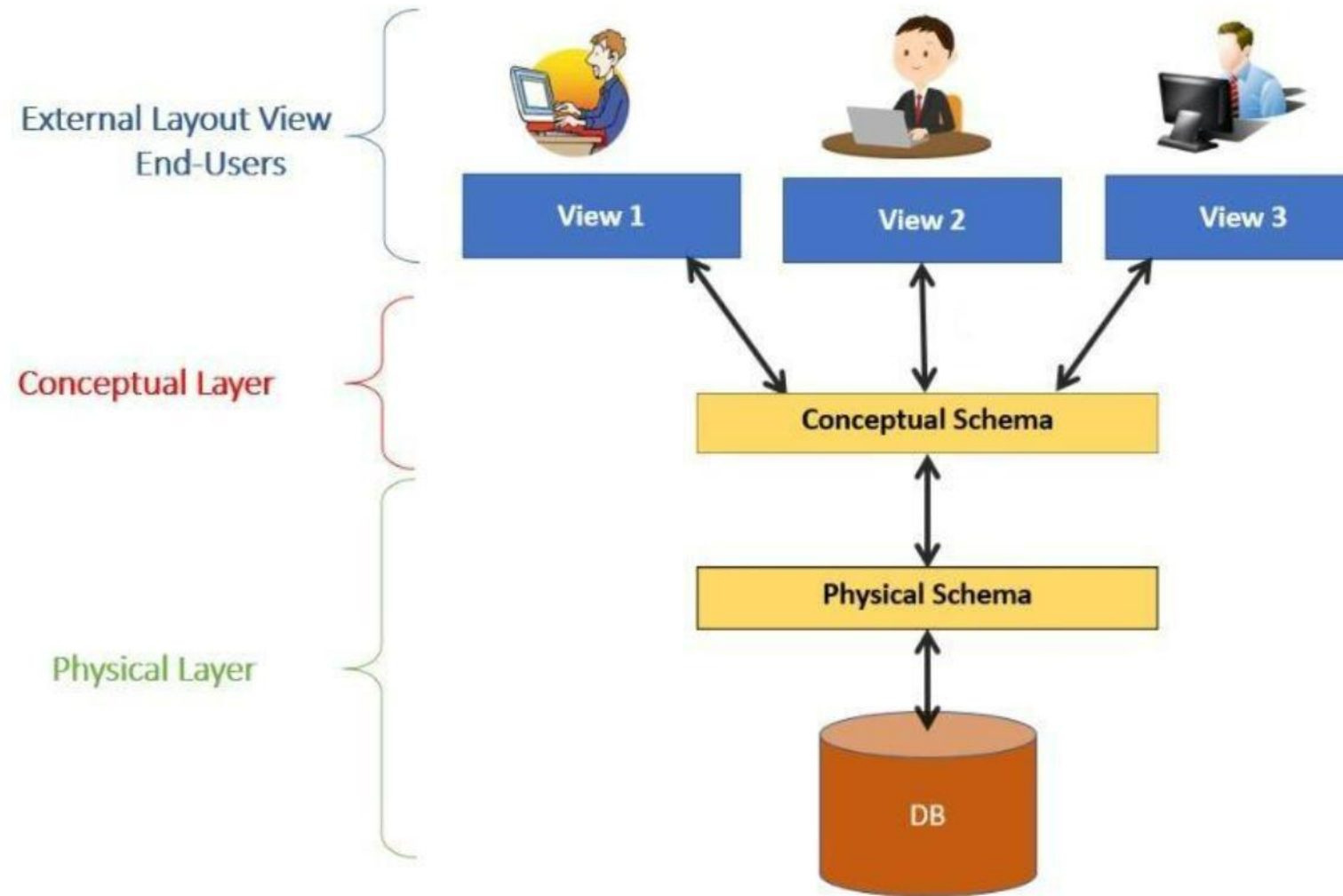
Three Schema Architecture in DBMS

The three-schema architecture divides the database into three-level used to create a separation between the physical database and the user application. In simple terms, this architecture hides the details of physical storage from the user.

Three Schema Architecture in DBMS



Three Schema Architecture in DBMS

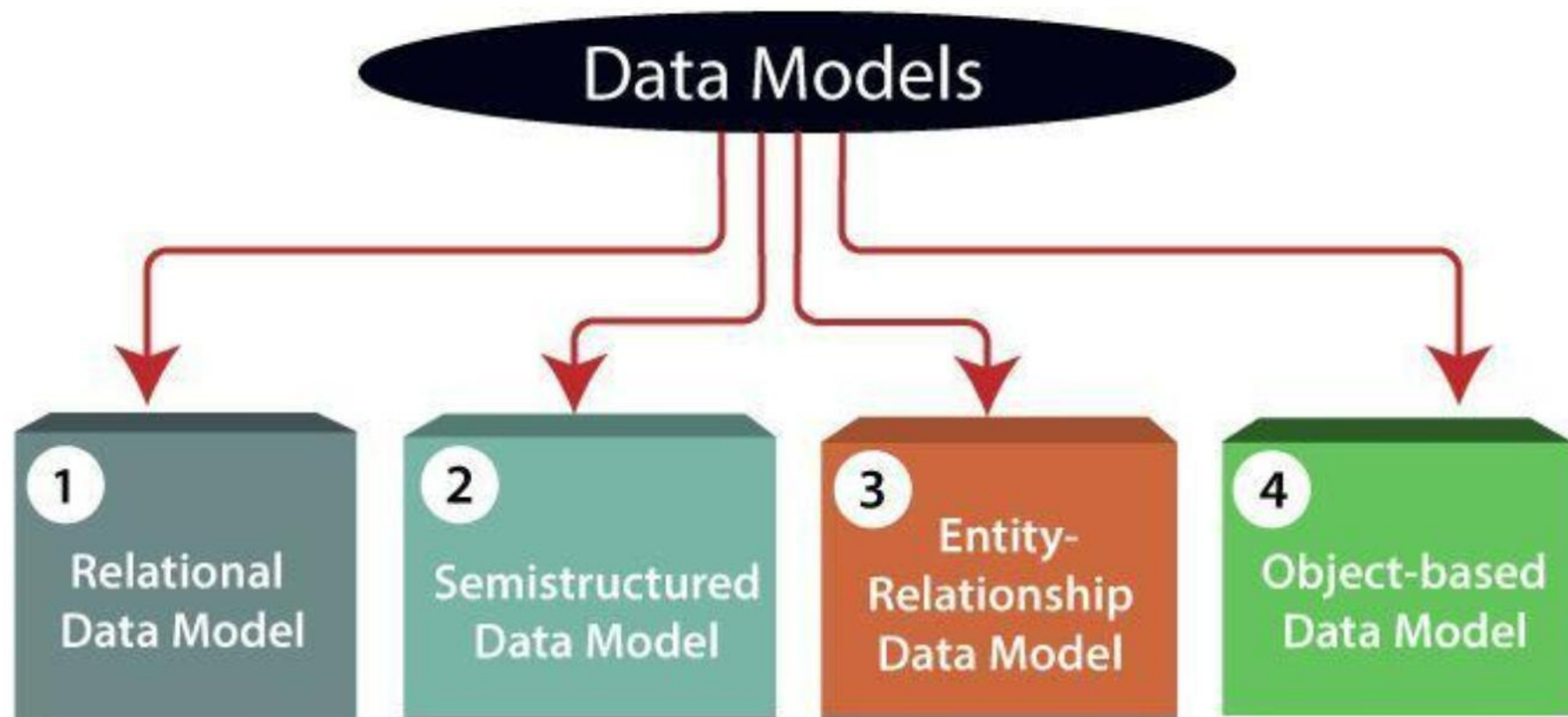


Levels of DBMS Architecture Diagram



Data Models

- Data models describe how a database's logical structure is represented. In a database management system.
- Data models are essential for introducing abstraction.
- Data models specify how data is linked to one another, as well as how it is handled and stored within the system.



Relational Data Model

- ✓ This type of model designs the data in the form of rows and columns within a table.
- ✓ Thus, a relational model uses tables for representing data and in-between relationships. Tables are also called relations.
- ✓ This model was initially described by Edgar F. Codd, in 1970.

Entity-Relationship Data Model

- ✓ An ER model is the logical representation of data as objects and relationships among them.
- ✓ These objects are known as entities, and relationship is an association among these entities.
- ✓ It was widely used in database designing.



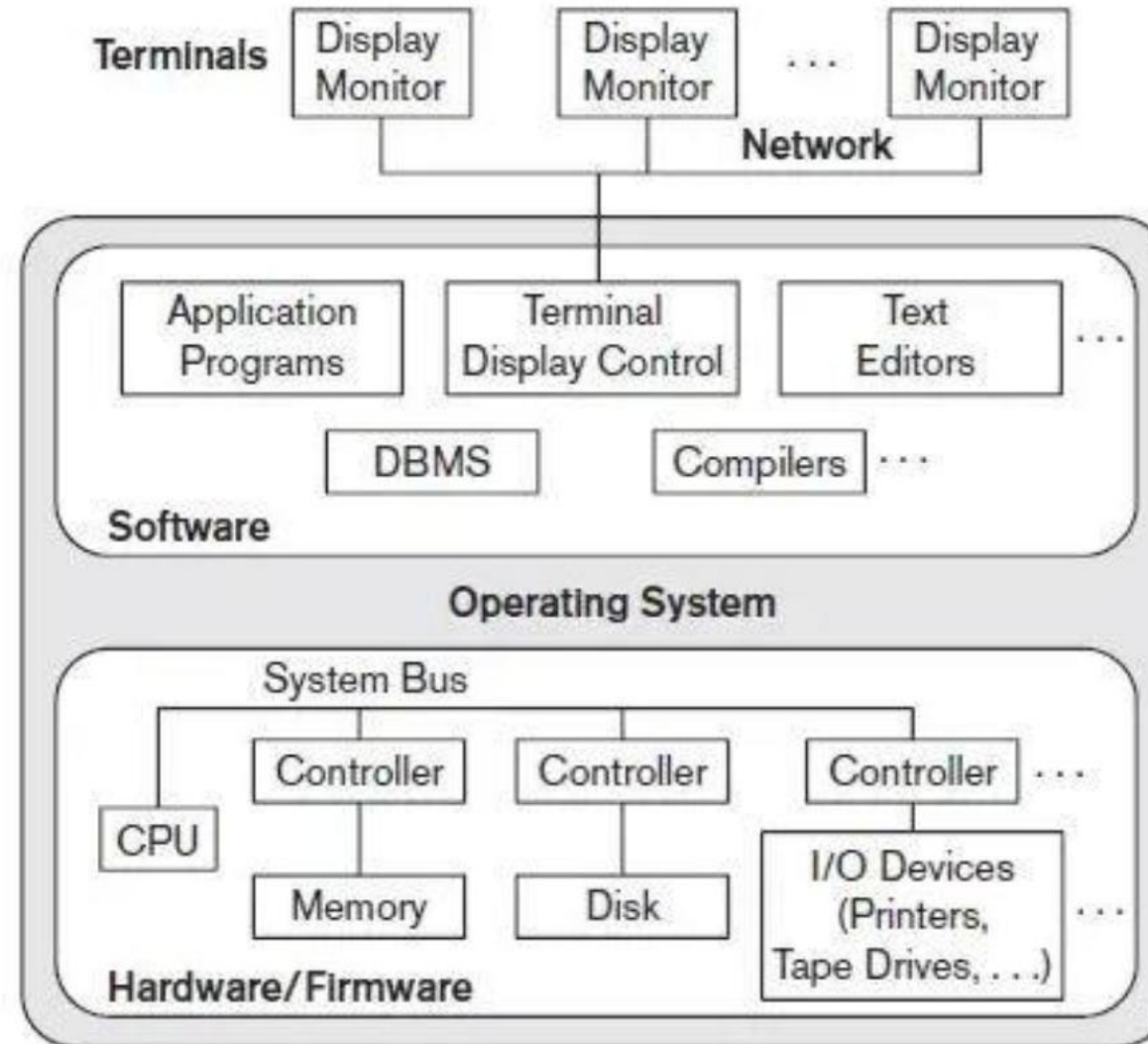
Object-based data model

- ✓ An extension of the ER model with notions of functions, encapsulation, and object identity, as well.
- ✓ This model supports a rich type system that includes structured and collection types.

Semi-structured data model

- ✓ This type of data model is different from the other three data models.
- ✓ The semistructured data model allows the data specifications at places where the individual data items of the same type may have different attributes sets.
- ✓ The Extensible Markup Language, also known as XML, is widely used for representing the semistructured data.

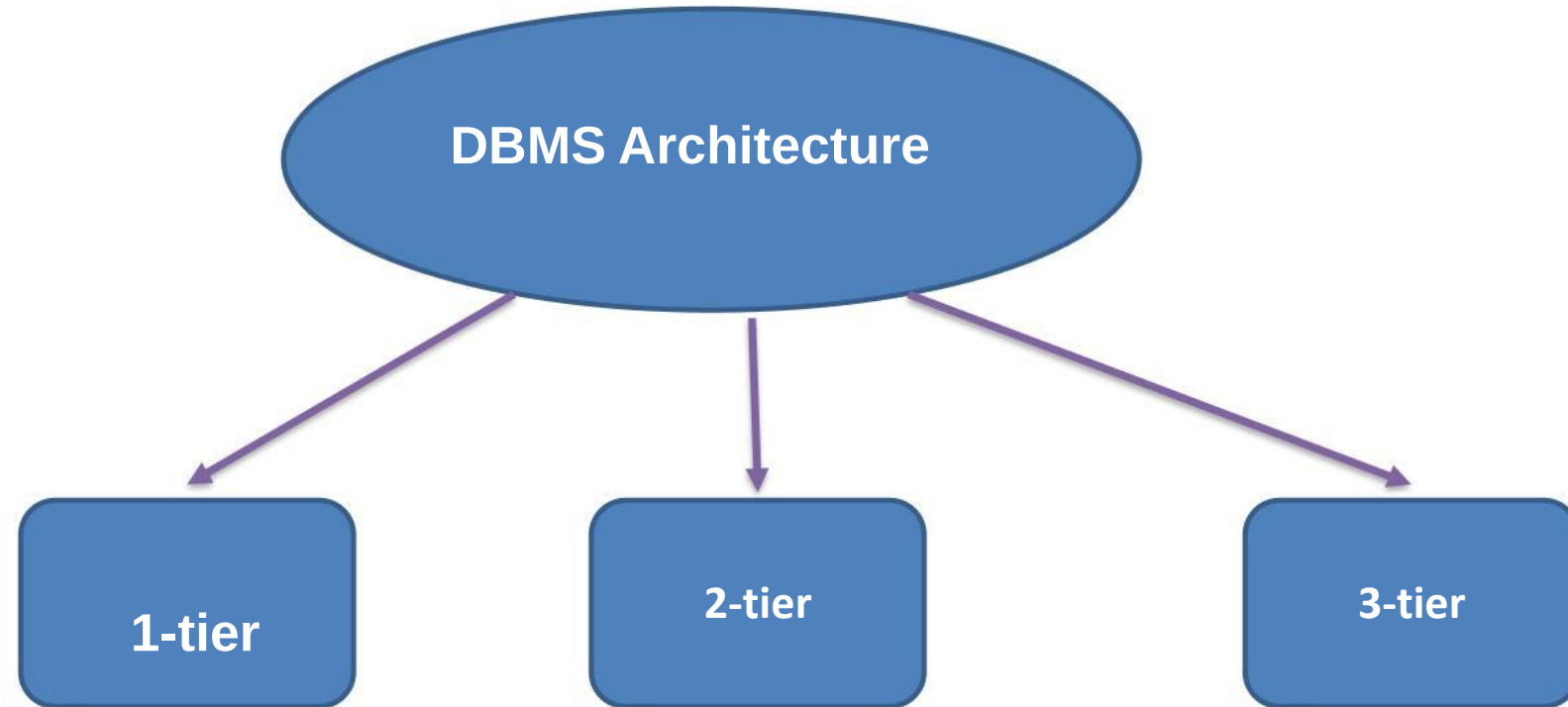
Centralized Architecture for DBMS



Centralized Architecture for DBMS

- ✓ Centralized databases are the traditional database systems where all database functionality, data, application program and user interface processing are located on one machine.
- ✓ In the early systems, the mainframe computers provided all the functionalities required by the users.
- ✓ The users interacted with the mainframe computers via dumb terminals. All processing was performed on the mainframe computer.
- ✓ The centralized database is easy to manage and administer.
- ✓ However, there are also some drawbacks to this type of architecture.
 - Higher data traffic
 - If there is a system failure users do not have access to the database

DBMS Client/Server Architecture



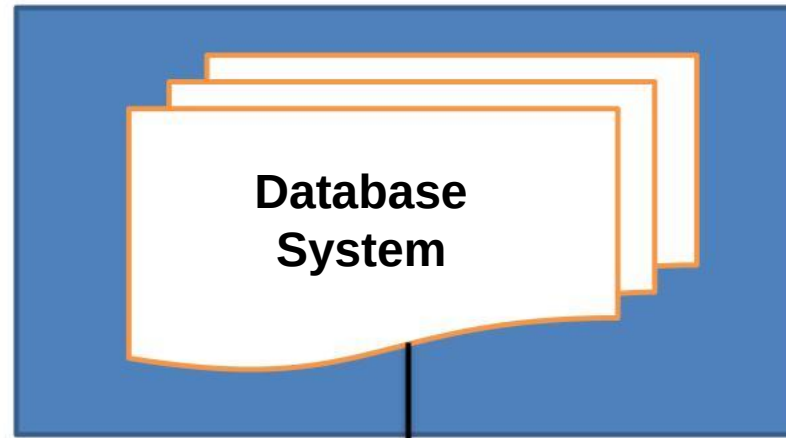
1-tier Architecture

- ✓ In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- ✓ Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- ✓ The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.
- ✓ A simple one tier architecture example would be anytime you install a Database in your system and access it to practice SQL queries. But such architecture is rarely used in production.

2-tierArchitecture

- ✓The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: ODBC, JDBC are used.
- ✓The user interfaces and application programs are run on the client-side.
- ✓The server side is responsible to provide the functionalities like: query processing and transaction management.
- ✓To communicate with the DBMS, client-side application establishes a connection with the server side.
- ✓A Contact Management System created using MS- Access.

Server



Client

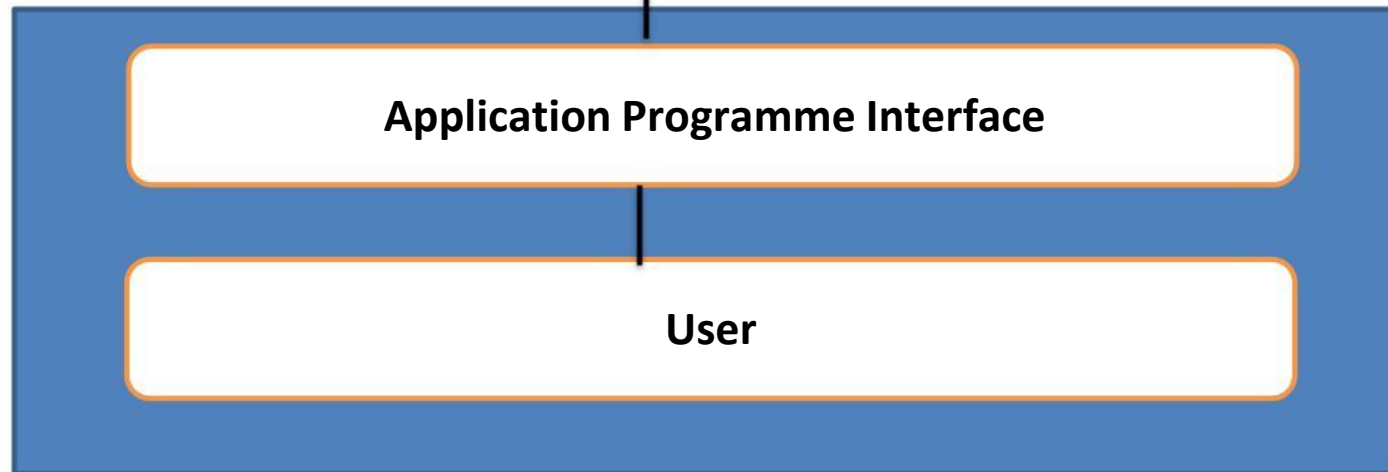


Fig: 2-tier architecture of DBMS

3-tier Architecture

- ✓ The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- ✓ The application on the client-end interacts with an application server which further communicates with the database system.
- ✓ End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- ✓ The 3-Tier architecture is used in case of large web application.
- ✓ Any large website on the internet.

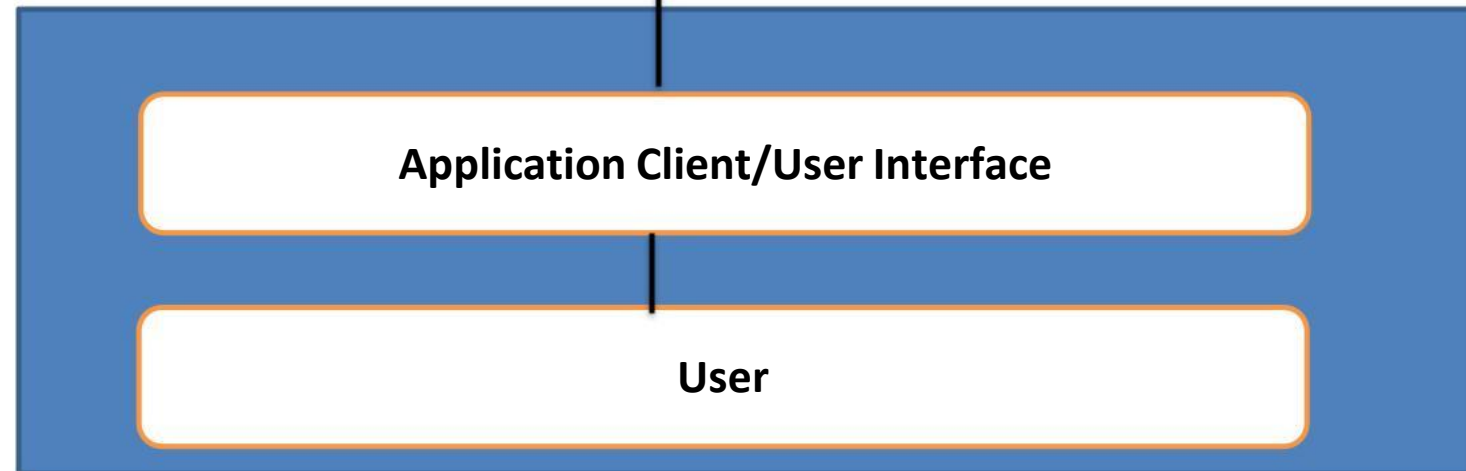
Server



Database System

Application Server

Client

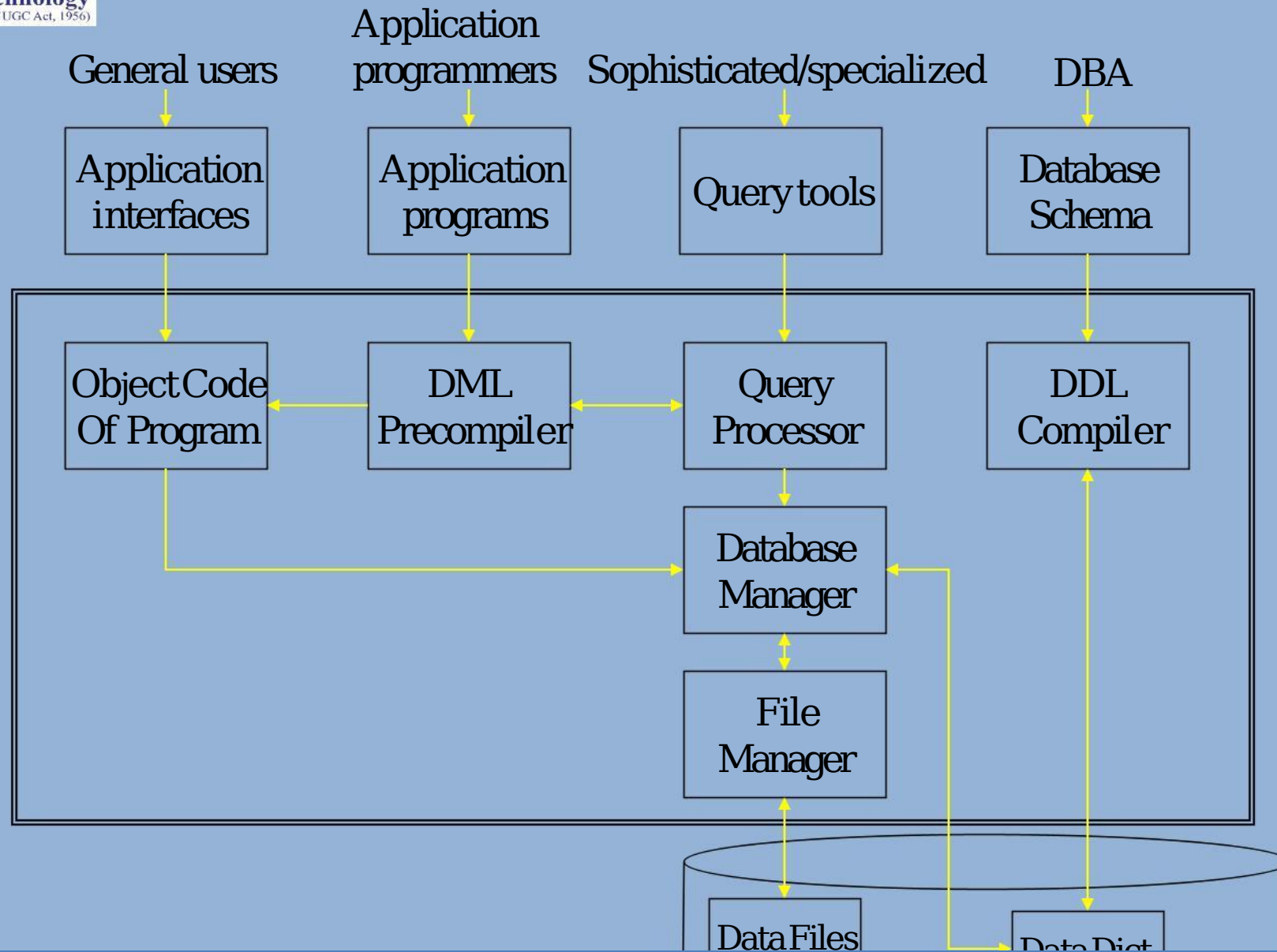


Application Client/User Interface

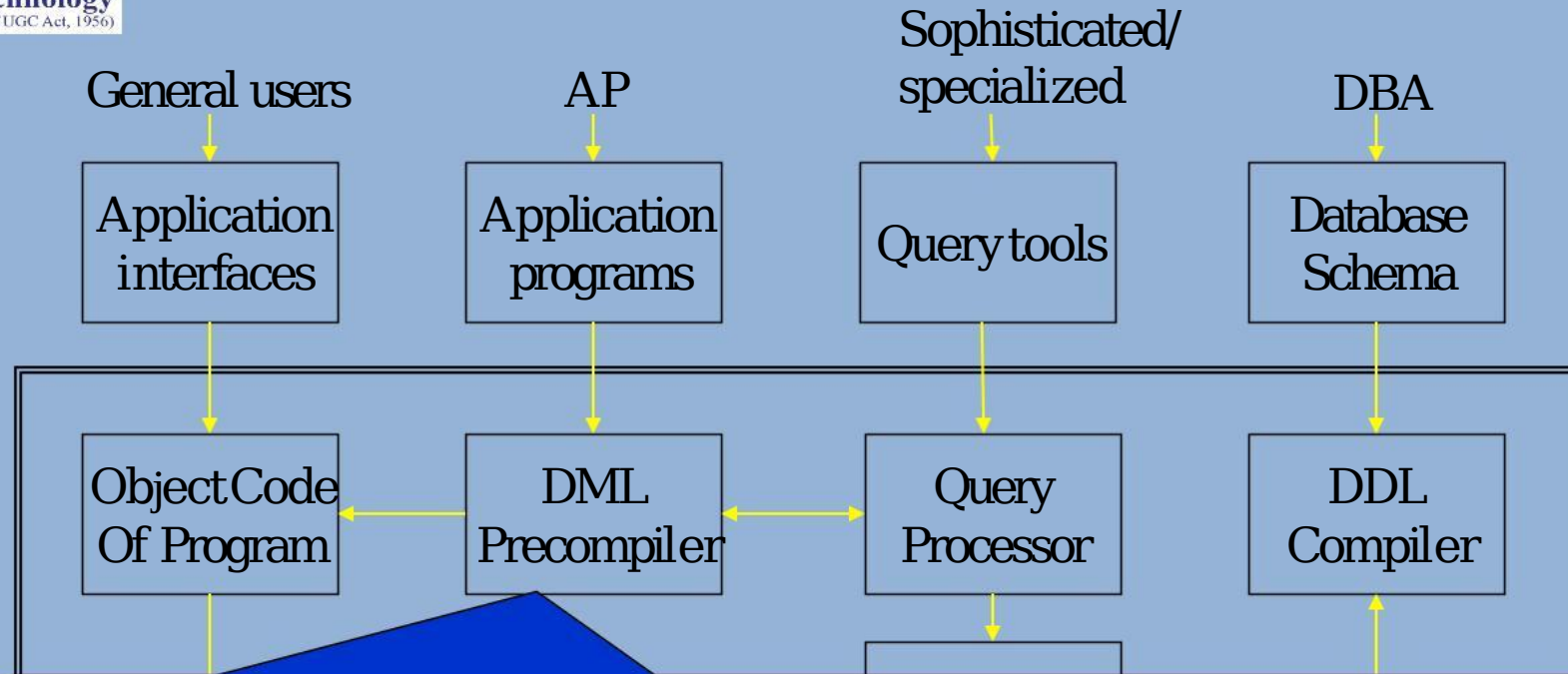
User

Fig: 3-tier architecture of DBMS

Overall DBMS Architecture

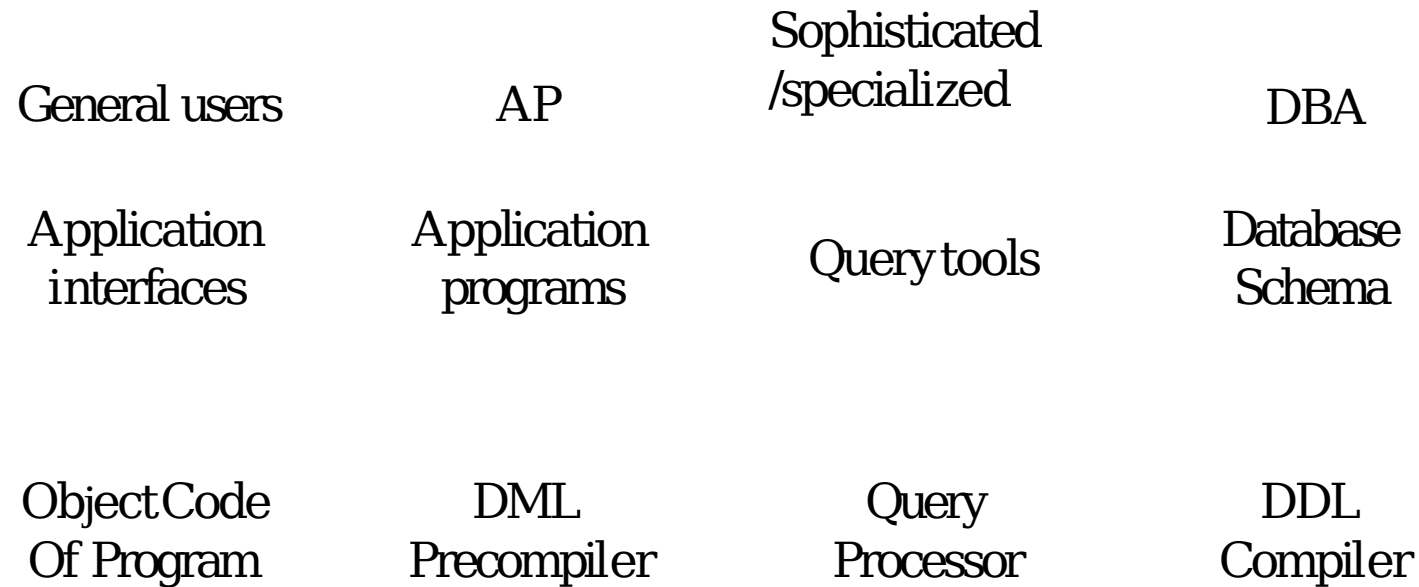


Overall DBMS Architecture



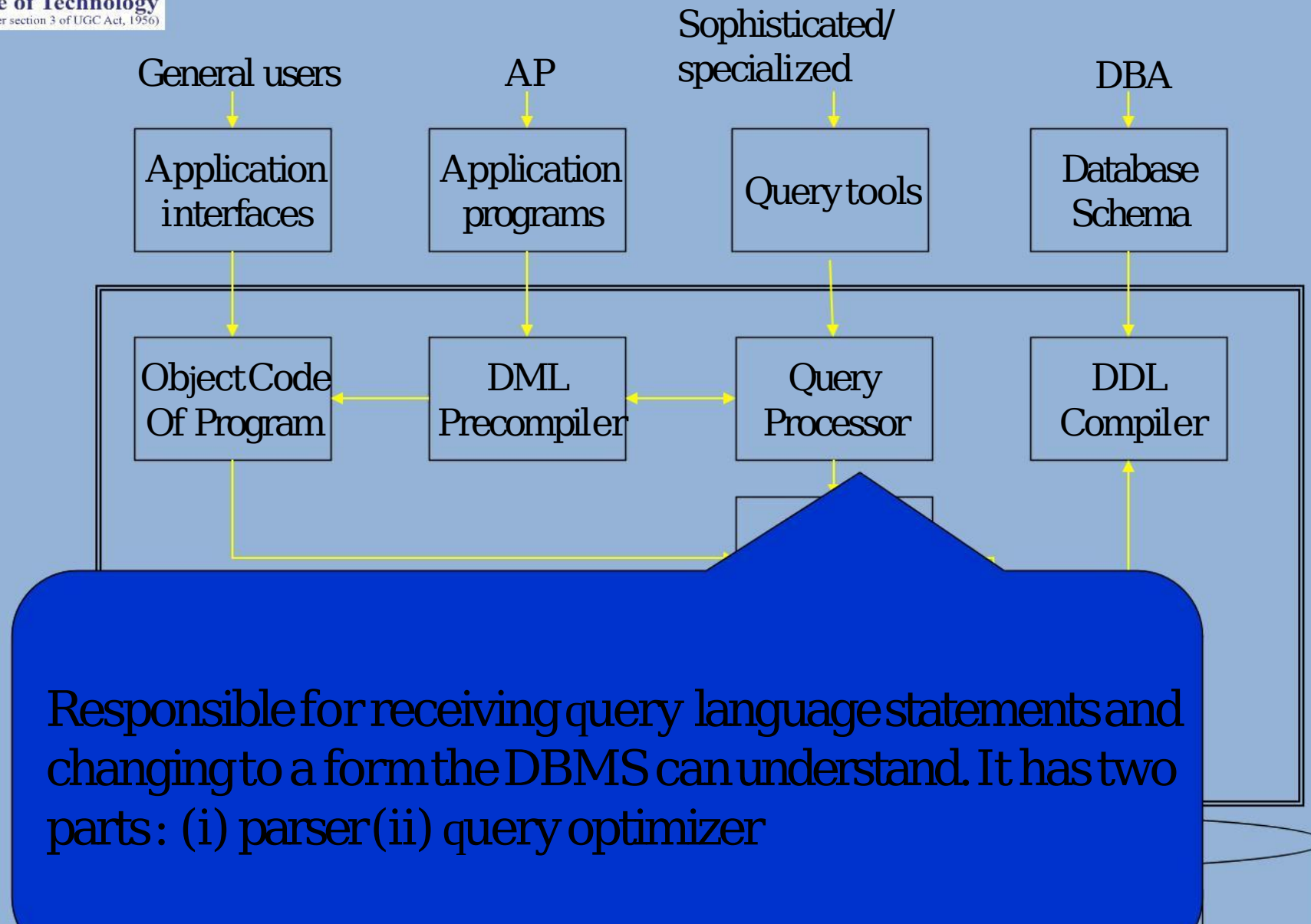
DDL– set of commands required to define the format of data.
 DML– set of commands that modify, process data.
 DML precompiler - It processes DML statements embedded in an application program into procedural calls.

Overall DBMS Architecture

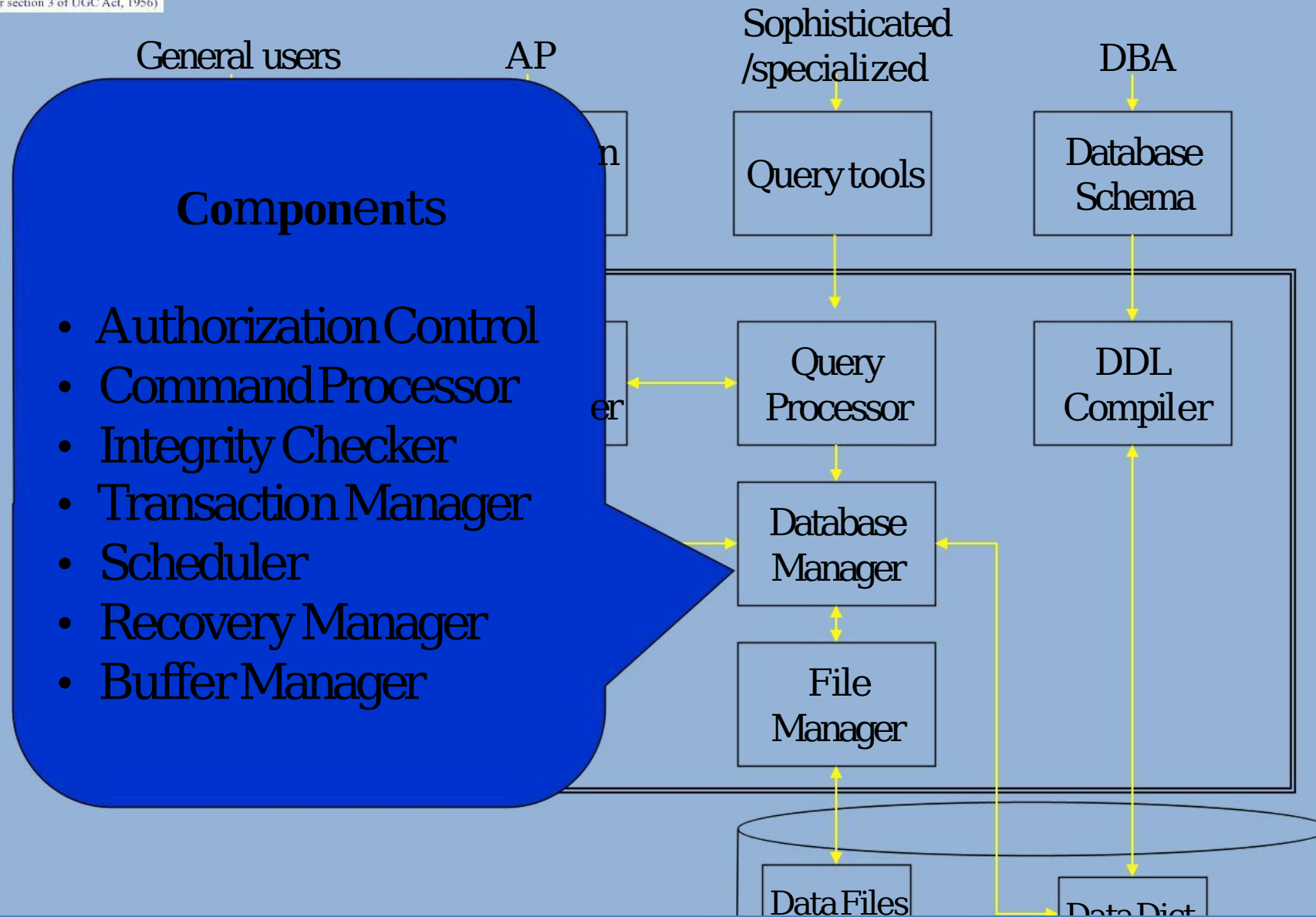


DDL compiler converts DDL statements into a set of tables containing metadata tables – which are in a form that can be used by other components of the DBMS. These are stored in system catalog or data dictionary.

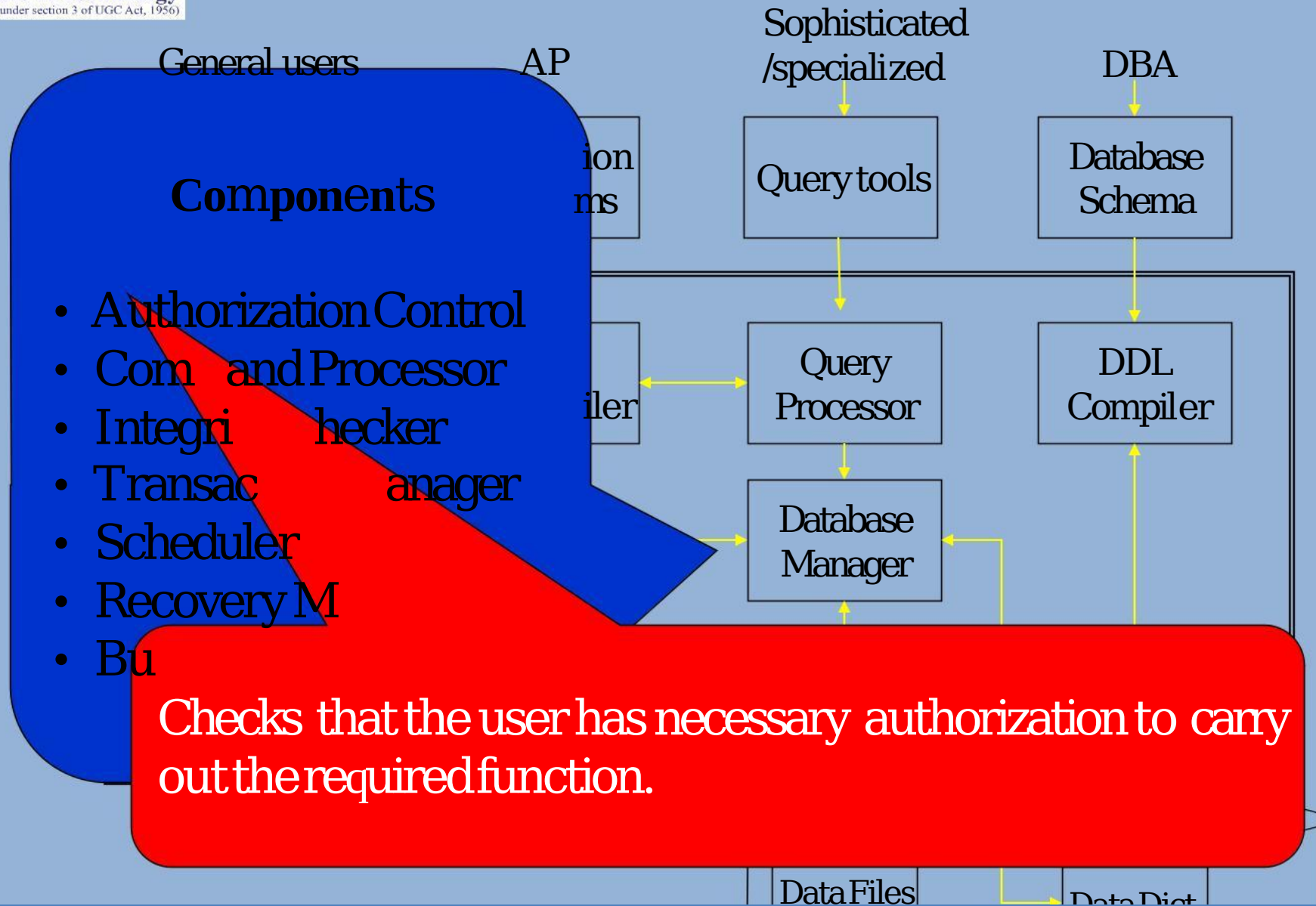
Overall DBMS Architecture



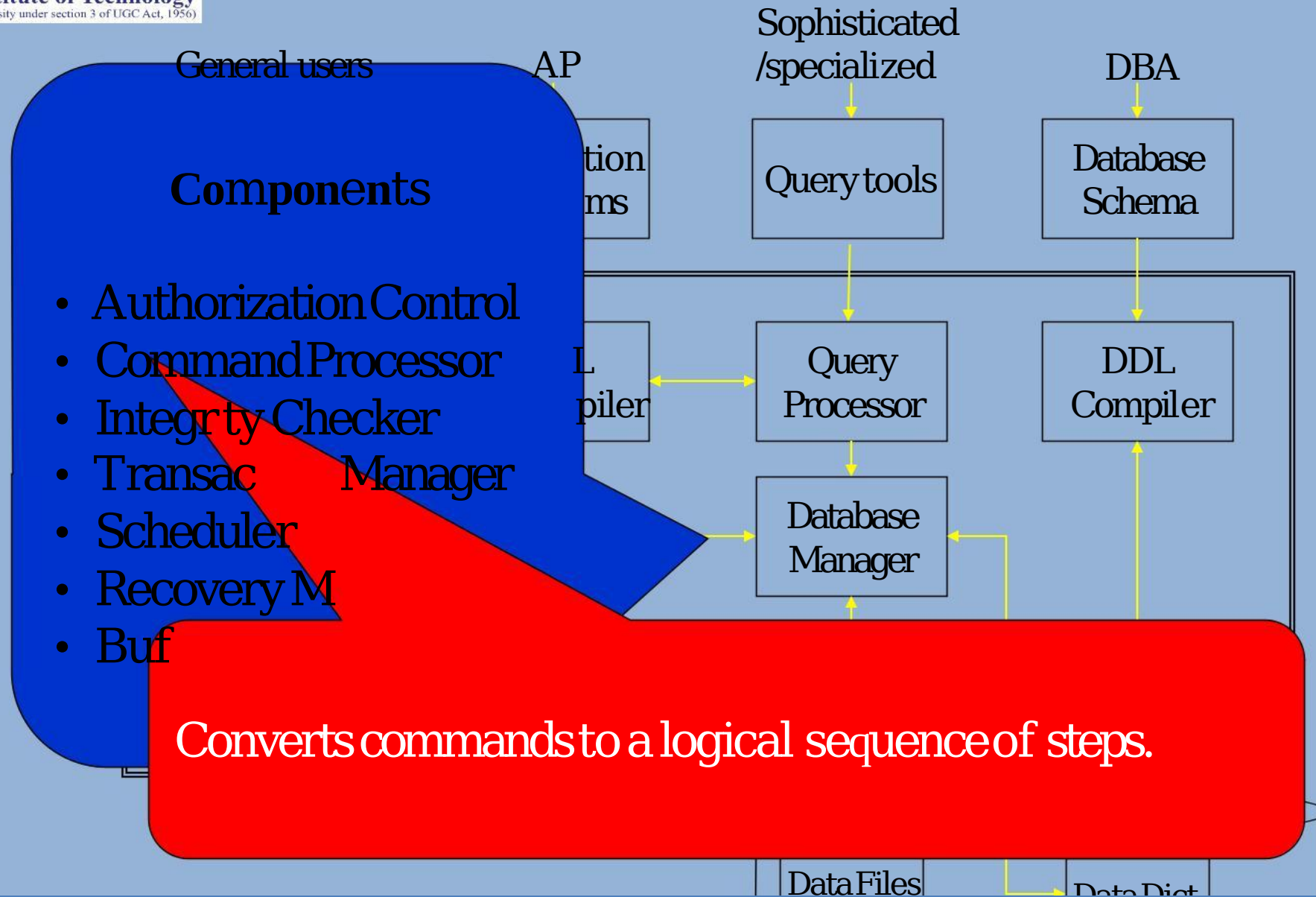
Overall DBMS Architecture



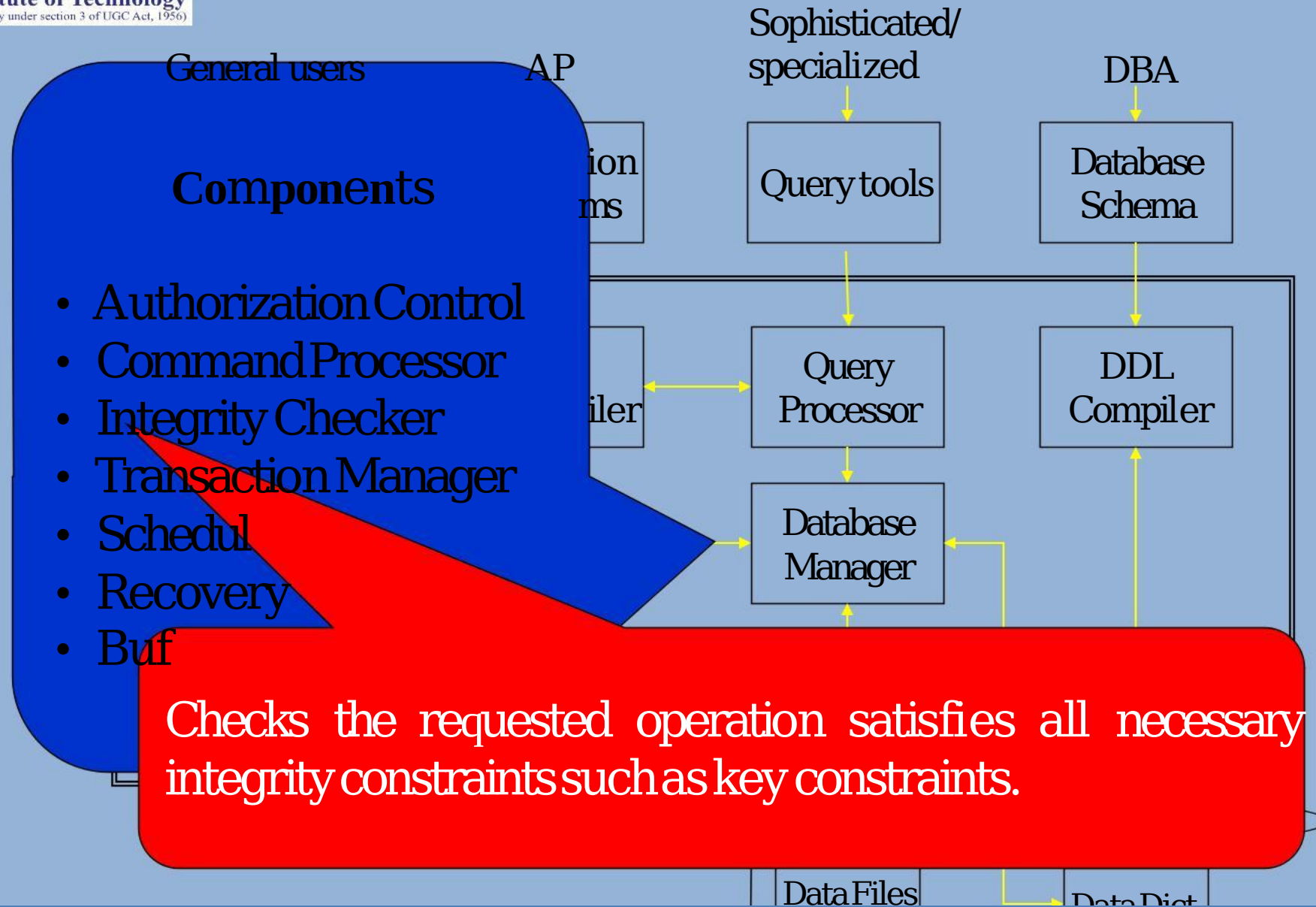
Overall DBMS Architecture



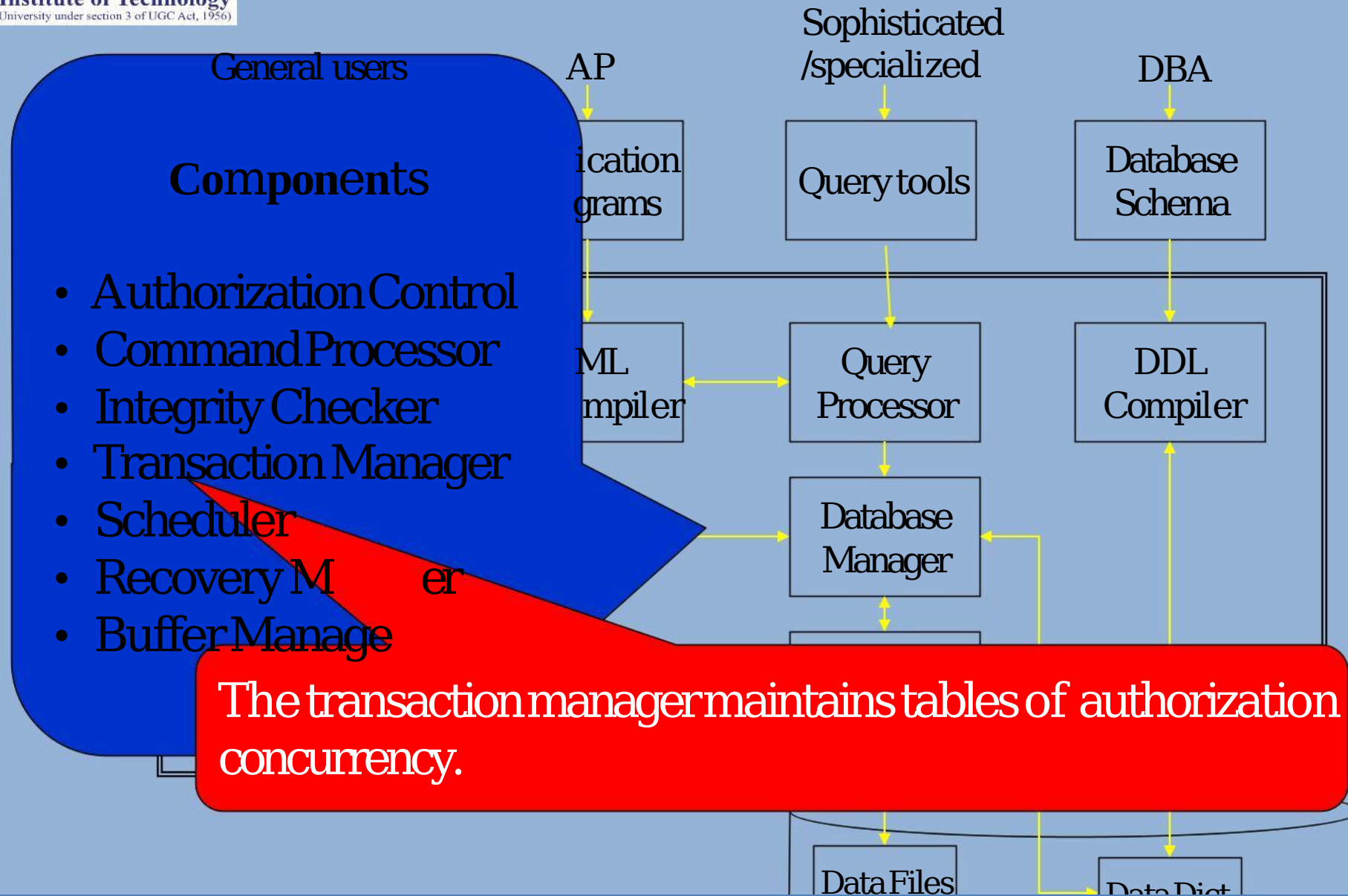
Overall DBMS Architecture



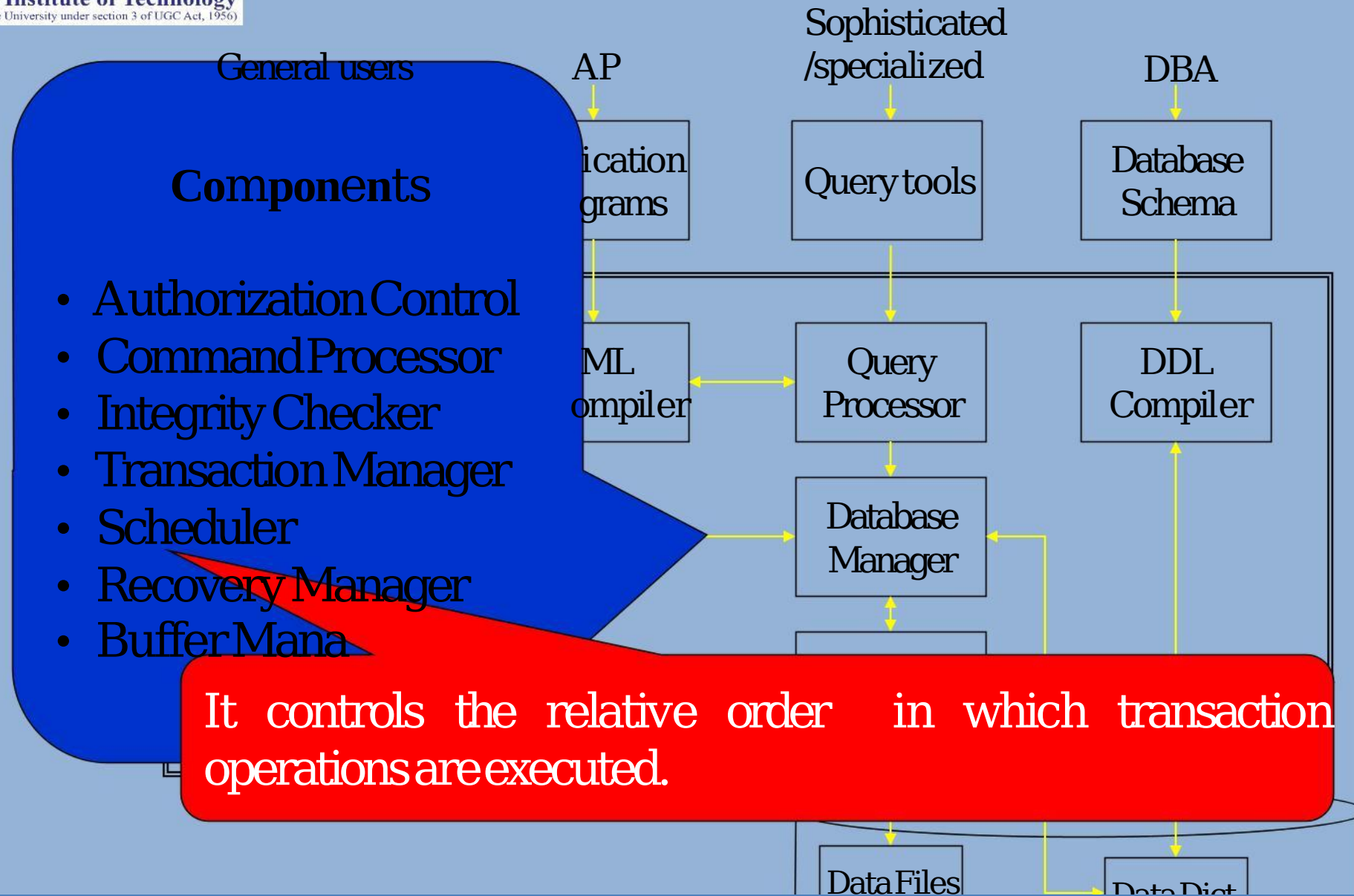
Overall DBMS Architecture



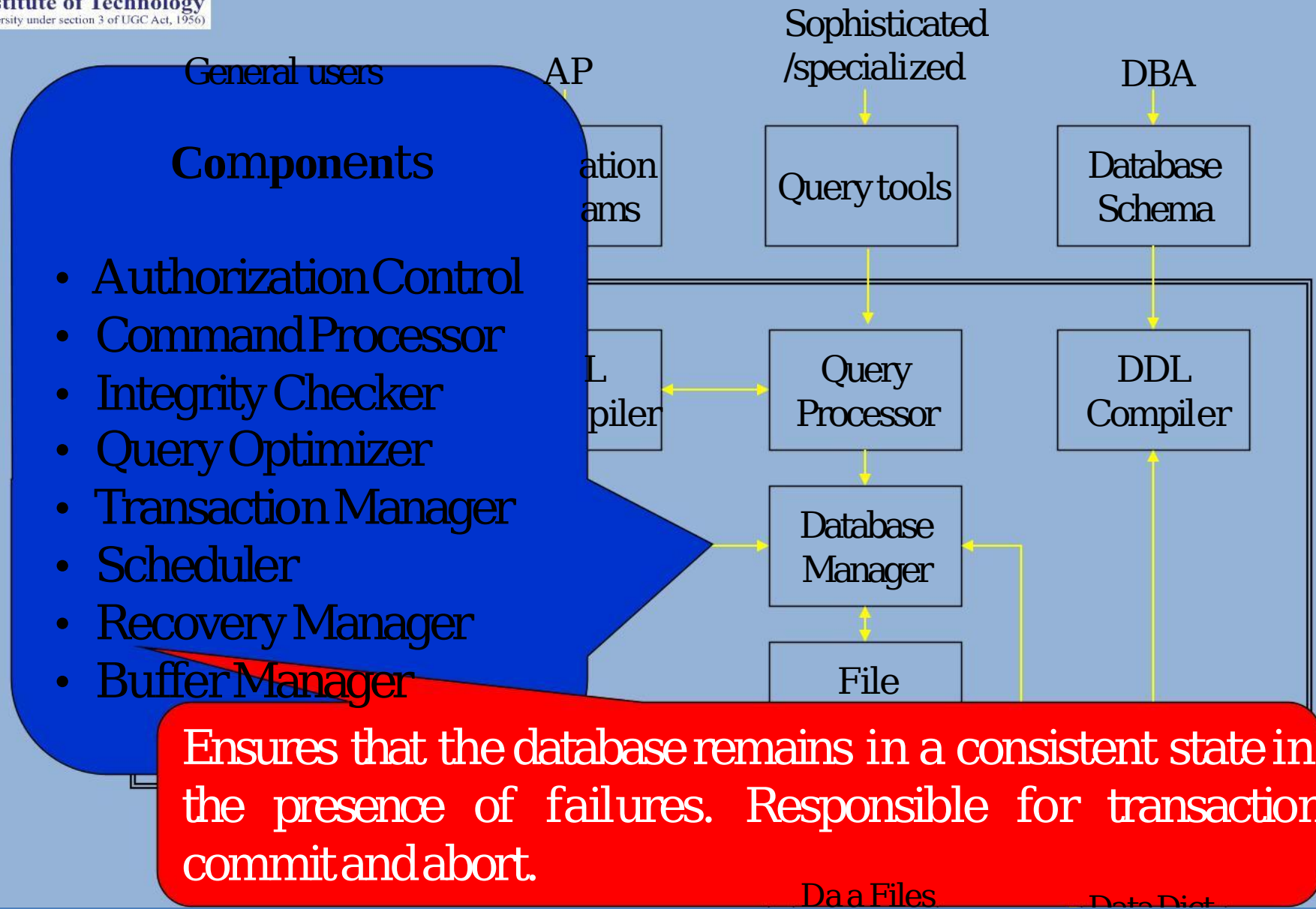
Overall DBMS Architecture



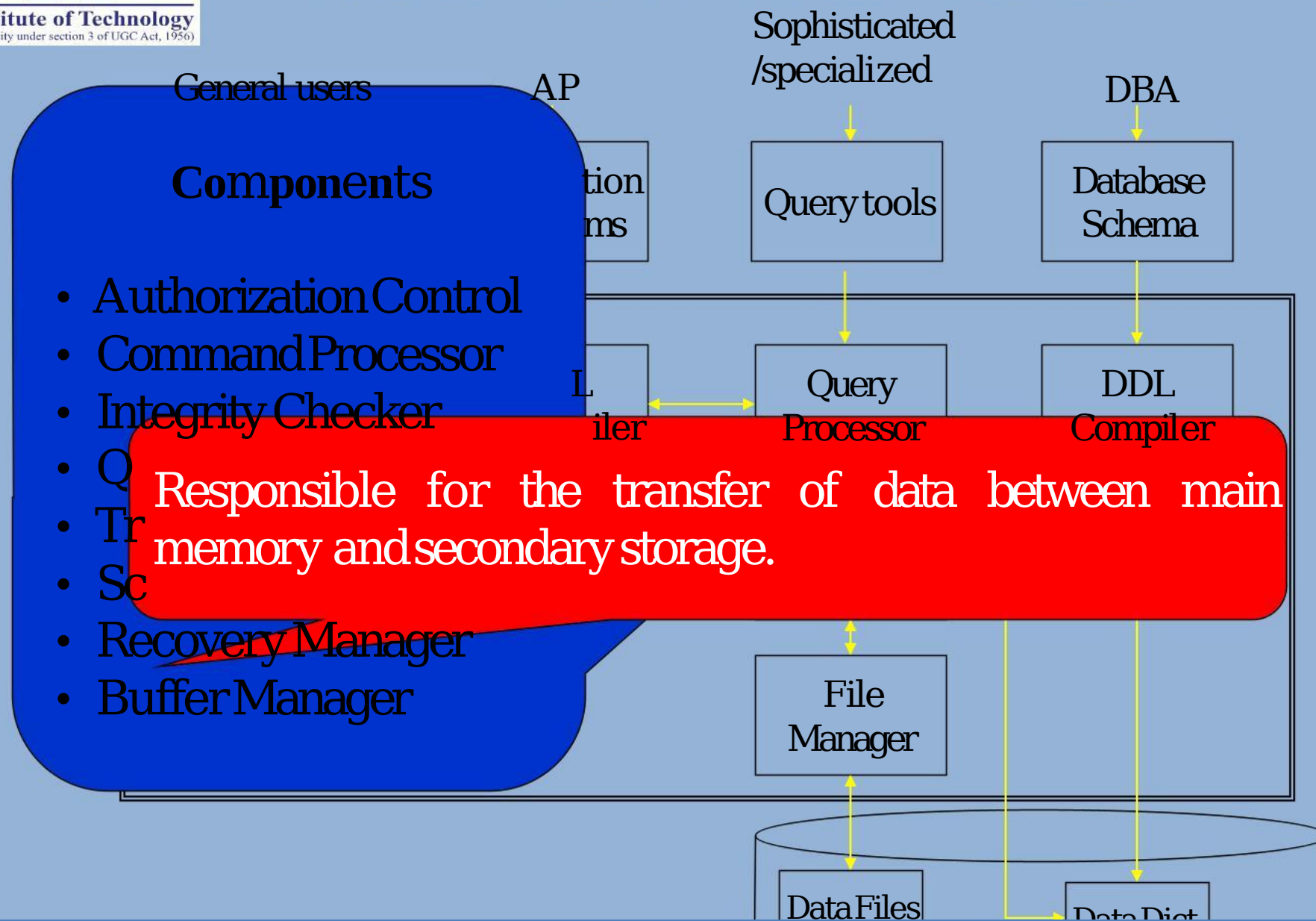
Overall DBMS Architecture



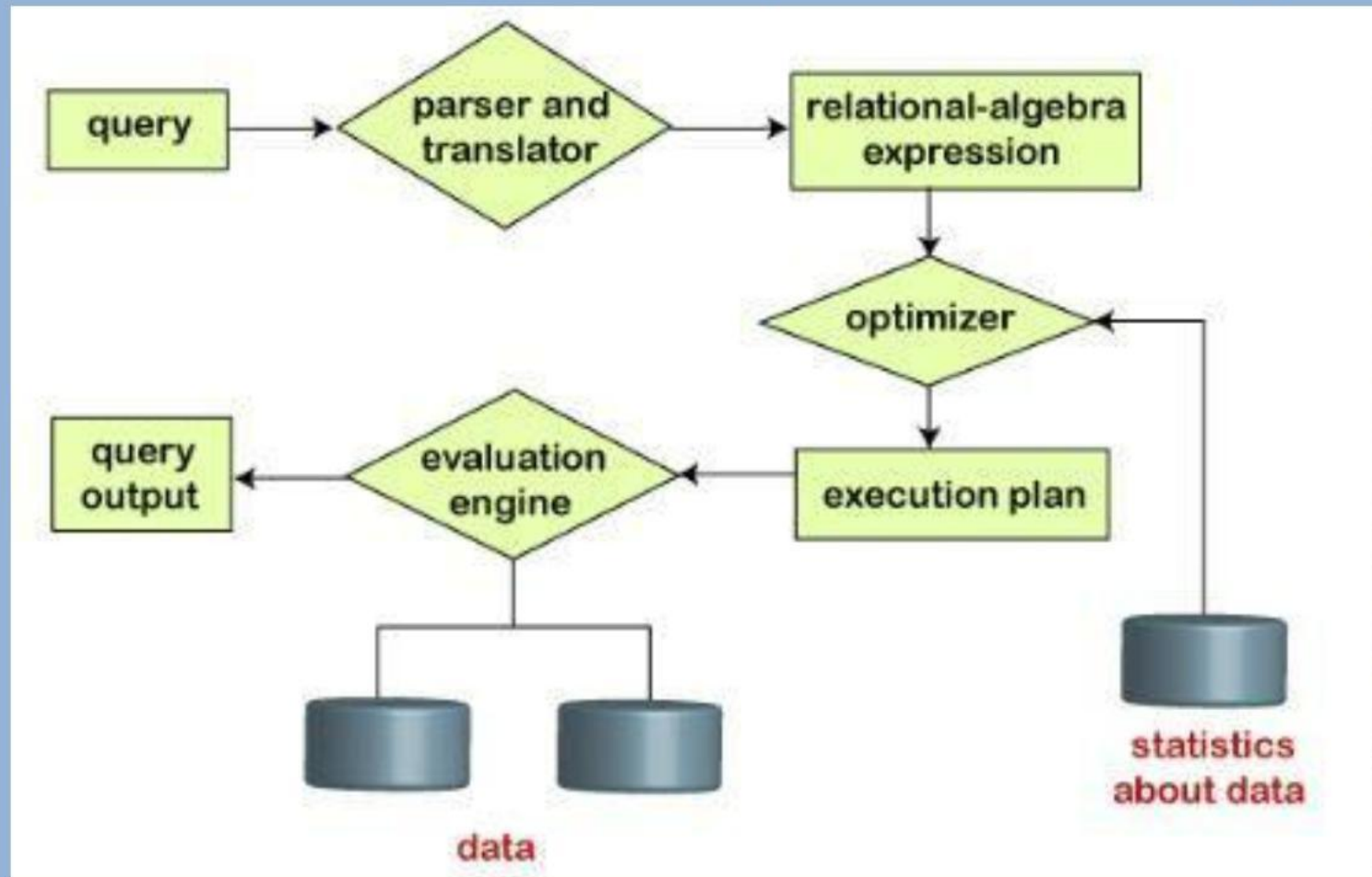
Overall DBMS Architecture



Overall DBMS Architecture



Procedure for Database Access/Query Processing



Database Languages

- SQL—a 4th generation language

Generation of Computer Languages	Examples	Code Sample
First	Machine code	0011011010
Second	Assembly language	MV R1 R2
Third	FORTRAN, COBOL, C, C++, Java, PHP, Perl, etc.	IF TRUE THEN ... END IF;
Fourth	SQL, assorted artificial intelligence languages	SELECT FIRST_NAME FROM EMPLOYEES;

- DDL (CREATE, ALTER, DROP, RENAME, TRUNCATE, COMMENT)
- DML (INSERT, UPDATE, DELETE)
- DCL (GRANT, REVOKE)
- TCL (COMMIT, ROLLBACK, SAVEPOINT)
- DQL (SELECT)



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Thank You!